ETSI GS ENI 059 V4.1.1 (2026-09)



GROUP SPECIFICATION

Experiential Networked Intelligence (ENI); AI Agent Interface and Protocol Specification for Next-Generation Mobile Communication System

Disclaimer

The present document has been produced and approved by the Experiential Networked Intelligence (ENI) ETSI Industry Specification Group (ISG) and represents the views of those members who participated in this ISG.
It does not necessarily represent the views of the entire ETSI membership.

Reference
DGS/ENI-0059_AIAI_PS4NGMCS

Keywords
AI Agent, AI-native, GenAI,
Next Generation Protocol

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed,
this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to
the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our
Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 6
Foreword 6
Modal verbs terminology 6
Executive summary 6
1 Scope 7
2 References 7
2.1 Normative references 7
.2.2 Informative references 8
3 Definition of terms, symbols and abbreviations 9
3.1 Terms 9
3.2 Symbols 9
3.3 Abbreviations 9
4 Background and Design Rationale 11
4.1 Motivation 11
4.2 Gap Analysis with Respect to Existing Protocols 11
4.3 Goals and Design Principles 12
4.4 Relationship to the ENI System Architecture 13
5 Interface Design 13
5.1 Definitions 13
5.2 Interface between Network AI Agent and UE 14
5.2.1 Requirements 14
5.2.2 Methods and Data Objects 15
5.2.3 Protocol 23
5.2.4 Security 24
5.2.4.1 Authentication 24
5.2.4.2 Authorization 24
5.2.4.3 Data Access Constraints 24
5.2.4.4 Transport Security 25
5.3 Interface between Network AI Agent and Third-Party Applications 25
5.3.1 Requirements 25
5.3.2 Methods and Data Objects 25
5.3.3 Protocol 27
5.3.4 Security 30
5.3.4.1 Authentication 30
5.3.4.2 Authorization 30
5.3.4.3 Data Access Constraints 30
5.3.4.4 Transport Security 30
5.4 Interface between Network AI Agent and Agent Repository 30
5.4.1 Requirements 30
5.4.2 Methods and Data Objects 31
5.4.3 Protocol 36
5.4.4 Security 36
5.4.4.1 Authentication 36
5.4.4.2 Authorization 36
5.4.4.3 Data Access Constraints 37
5.4.4.4 Transport Security 38
5.5 Interface among Network AI Agents 38
5.5.1 Requirements 38
5.5.2 Methods and Data Objects 38
5.5.3 Protocol 45
5.5.4 Security 45
5.5.4.1 Authentication 45
5.5.4.2 Authorization 45

5.5.4.3 Data Access Constraints....46
5.5.4.4 Transport Security....46
5.6 Interface between Network AI Agent and Resources....46
5.6.1 Interface between Network AI Agent and Data....46
5.6.1.1 Requirements....46
5.6.1.2 Methods and Data Objects....47
5.6.1.3 Protocol....50
5.6.2 Interface between Network AI Agent and Tools....51
5.6.2.1 Requirements....51
5.6.2.2 Methods and Data Objects....51
5.6.2.3 Protocol....55
5.6.3 Security....55
5.6.3.1 Authentication....55
5.6.3.2 Authorization....55
5.6.3.2.0 Introduction....55
5.6.3.2.1 Agent-to-Data Interface....55
5.6.3.2.2 Agent-to-Tool Interface....56
5.6.3.3 Data Access Constraints....57
5.6.3.4 Transport Security....57
5.7 Interface between Network AI Agent and Infrastructure....57
5.7.1 Interface between Network AI Agent and RAN....57
5.7.1.1 Requirements....57
5.7.1.2 Methods and Data Objects....57
5.7.1.3 Protocol....61
5.7.2 Interface between Network AI Agent and Computing Infrastructure....61
5.7.2.1 Methods and Data Objects....61
5.7.2.2 Agent-to-Computing Protocol Requirements....61
5.7.2.3 Agent-to-Computing Security Requirements....61
5.7.3 Security....62
5.7.3.1 Authentication....62
5.7.3.2 Authorization....62
5.7.3.2.0 Introduction....62
5.7.3.2.1 Agent-to-RAN Interface....62
5.7.3.2.2 Agent-to-Computing-Infrastructure Interface....63
5.7.3.3 Data Access Constraints....63
5.7.3.4 Transport Security....63
5.8 Security Requirements....63
5.8.1 Introduction....63
5.8.2 Authentication....63
5.8.2.1 General....63
5.8.2.2 UE Authentication....63
5.8.2.3 Third-Party Application Function (AF) Authentication....64
5.8.2.4 Network AI Agent Authentication....64
5.8.2.5 Data and Tool Server Authentication....64
5.8.3 Authorization....64
5.8.3.1 General....64
5.8.3.2 AF Authorization....64
5.8.3.3 Network AI Agent Authorization....65
5.8.3.4 UE Authorization....65
5.8.3.5 Data Sensitivity Classes....65
5.8.4 Transport Security....65
5.8.5 Access Control for the Agent Repository Function....65
5.8.6 Auditability and Traceability....66
5.8.7 Relationship to Referenced Security Frameworks....66
6 Communication Examples....66
6.1 Agent Interface Communication Examples....66
6.2 Agent-to-Tool Interaction Example....66
6.3 Agent-to-Data Interaction Example....68
6.4 Agent-to-Agent Interaction Example....69
7 Areas for Future Study....72

7.1 Open issues for the present document 72
7.2 Issues for Future Study 72
History 74

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Group Specification (GS) has been produced by ETSI Industry Specification Group (ISG) Experiential Networked Intelligence (ENI).

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

The present document specifies the interface and protocol design for AI Agent communication in Next-Generation Mobile Communication Systems (NGMCS). It identifies gaps in existing agent communication protocols (e.g. A2A, MCP) for use in telecommunications applications and defines a new set of methods and data objects for six distinct interface types: Network AI Agent to UE, to Third-Party Application, to Agent Repository Function (ARF), among Network AI Agents, to Data Resources, and to Tool Resources. The present document additionally specifies extensions to the NGAP protocol for AI agent interaction with RAN infrastructure, and includes protocol binding analysis covering HTTP/1.1+JSON-REST, HTTP/1.1+JSON-RPC 2.0, and HTTP/2+gRPC, with a recommendation for HTTP/3+gRPC.

1 Scope

The present document specifies interface and protocol design for enabling communication between AI Agents and the following entities in a mobile network: other AI agents, external tools and data sources, an Agent Repository Function for centralized AI agent lifecycle management, User Equipment (UEs), third-party application functions, and network infrastructure (e.g. Radio Access Network nodes and computing platforms).

In addition, the present document analyses two prominent agent communication protocols from a telecommunications point-of-view. It identifies a set of functional gaps based on this analysis that motivate the interface extensions defined herein. For each specified interface type, the present document defines the required methods, data objects, and protocol bindings applicable within an agent-based mobile network architecture. Communication examples are provided to illustrate the specified interface interactions.

The present document does not specify modifications to existing 3GPP NAS or NGAP procedures. Where such extensions are identified as necessary to realize the interfaces defined herein, the detailed procedure specification is deferred to the relevant 3GPP specifications and is outside the present document's normative scope. Conformance to 3GPP TS 24.501 [2] and ETSI TS 138 413 [11] is governed by those specifications.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

Referenced documents which are not found to be publicly available in the expected location might be found in the ETSI docbox.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

[1] 3GPP TS 23.501 (V19.0.0): "System architecture for the 5G System (5GS)".

[2] 3GPP TS 24.501 (V19.6.1): "Non-Access-Stratum (NAS) protocol for 5G System (5GS)".

[3] JSON-RPC Working Group: "JSON-RPC 2.0 Specification", January 2013.

[4] ETSI GS ENI 005 (V4.1.1): "Experiential Networked Intelligence (ENI); ENI System Architecture".

[5] 3GPP TS 33.501 (V20.1.0): "Security architecture and procedures for 5G system (Release 20)".

[6] The Semantic Versioning 2.0.0.

[7] IETF RFC 3986: "Uniform Resource Identifier (URI): Generic Syntax".

[8] ETSI GS ENI 019 (V3.1.1): "Experiential Networked Intelligence (ENI); Representing, Inferring, and Proving Knowledge in ENI".

[9] ETSI TS 123 003 (V19.6.0): "Digital cellular telecommunications system (Phase 2+) (GSM); Universal Mobile Telecommunications System (UMTS); LTE; 5G; Numbering, addressing and identification (3GPP TS 23.003 version 19.6.0 Release 19)".

[10] IETF RFC 2046: "Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types".

[11] ETSI TS 138 413 (V19.2.0): "5G; NG-RAN; NG Application Protocol (NGAP) (3GPP TS 38.413 version 19.2.0 Release 19)".

[12] IETF RFC 8259: "The JavaScript Object Notation (JSON) Data Interchange Format".

[13] ETSI TS 123 222 (V19.9.0): "LTE; 5G; Common API Framework for 3GPP Northbound APIs (3GPP TS 23.222 version 19.9.0 Release 19)".

[14] ISO 8601: "Date and time — Representations for information exchange".

.2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] 3GPP TR 22.870 (V20.0.0): "Study on 6G Use Cases and Service Requirements".

[i.2] ETSI GR ENI 055 (V4.1.1): "Experiential Networked Intelligence (ENI); Use Cases and Requirements for AI Agents based Core Network".

[i.3] ETSI GR ENI 051 (V4.1.1): "Experiential Networked Intelligence (ENI); Study on AI Agents based Next-generation Network Slicing".

[i.4] ETSI GR ENI 056 (V4.1.1): "Experiential Networked Intelligence (ENI); Study on Multi-Agent Frameworks for Next-Generation Core Networks".

[i.5] ETSI GR ENI 004 (V4.1.1): "Experiential Networked Intelligence (ENI); ENI terminology".

[i.6] IETF RFC 7541: "HPACK: Header Compression for HTTP/2".

[i.7] The A2A Protocol.

NOTE: The A2A Protocol is a Linux Foundation project. Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries.

[i.8] Xinyi Hou, Yanjie Zhao, Shenao Wang and Haoyu Wang (2025): "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions".

[i.9] M.R. Genesereth and S.P. Ketchpel: "Software Agents", Communications of the ACM, 37(7), pp. 48-53, 1994.

[i.10] Foundation for Intelligent Physical Agents: "FIPA Communicative Act Library Specification", 2001.

[i.11] W3C® Recommendation 30 April 2013: "PROV-DM: The PROV Data Model".

[i.12] NIST AI 100-1 (January 2023): "Artificial Intelligence Risk Management Framework (AI RMF 1.0)".

[i.13] Void.

[i.14] F. Bellifemine, A. Poggi and G. Rimassa: "JADE - A FIPA-compliant agent framework".

[i.15] IETF RFC 9170: "Long-Term Viability of Protocol Extension Mechanisms".

[i.16] D. Lübke, O. Zimmermann, C. Pautasso, U. Zdun and M. Stocker: "Interface Evolution Patterns — Balancing Compatibility and Extensibility across Service Life Cycles", 24th European Conference on Pattern Languages of Programs, 2019.

[i.17] NIST SP 800-53 (Revision 5 - September 2020): "Security and Privacy Controls for Information Systems and Organizations".

[i.18] NIST SP 800-207 (August 2020): "Zero Trust Architecture".

[i.19] NIST SP 800-52 (Revision 2 - August 2019): "Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations".

[i.20] IETF RFC 9204 (June 2022): "QPACK: Field Compression for HTTP/3".

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the terms given in ETSI GS ENI 005 [4], ETSI GR ENI 004 [i.5] and the following apply:

AI Agent: automated intelligent entity that performs reasoning, self-learning, decision-making, and task execution to achieve a specific goal

Agent Repository Function (ARF): network function responsible for storing and maintaining lifecycle information about registered Network AI agents

NOTE 1: Lifecycle information includes, but is not limited to, capabilities, endpoints, security schemes, and supported methods.

NOTE 2: The ARF enables an AI Agent to discover other AI Agents.

beyond-connectivity services: services that use the network as a compute/intelligence/sensing platform rather than a passive transport medium

Network AI Agent: AI Agent that resides in the mobile network that provides or supports providing network services

skill: self-contained, reusable unit of functionality comprised of instructions, required resources, and tool interfaces enabling an AI agent to perform a specific task or class of tasks

UE AI Agent: AI Agent that runs on a user equipment that supports intelligent and multimodal communication between the user and the mobile network

3.2 Symbols

Void.

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

5G	5th Generation
5QI	5G QoS Identifier
6G	6th Generation
A2A	Agent2Agent
AF	Application Function
AI	Artificial Intelligence
AIP	API Improvement Proposals
AMBR	Aggregated Maximum Bit Rate
AMF	Access and Mobility Management Function
API	Application Programming Interface
ARF	Agent Repository Function
ARP	Allocation and Retention Protocol
ASN	Abstract Syntax Notation
CAPIF	Common API Framework
CM	Conditional Mandatory

CN	Core Network
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, and Delete
DID	Decentralized Identifier
DRX	Discontinuous Reception
ERP	External Reference Point
FFS	For Further Study
FLOPS	Floating-Point Operations Per Second
GAN	Generative Adversarial Network
GFBR	Guaranteed Flow Bit Rate
gRPC	gRPC Remote Procedure Call

NOTE: This abbreviation is explained at https://grpc.github.io/grpc/core/md_doc_g_stands_for.html.

GUTI	Globally Unique Temporary Identifier
HPACK	Header Compression for HTTP/2

NOTE: This is defined in IETF RFC 7541 [i.6].

HTTP	Hypertext Transfer Protocol
ID	Identifier
IMEI	International Mobile Equipment Identity
IMSI	International Mobile Subscriber Identity
IP	Internet Protocol
ISO	International Organization for Standardization
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
MFBR	Maximum Flow Bit Rate
MIME	Multipurpose Internet Mail Extensions
MP4	MPEG-4 file format
mTLS	mutual Transport Layer Security
NA	Not Applicable
NAI	Network Access Identifier
NAS	Non-Access Stratum
NEF	Network Exposure Function
NF	Network Function
NGAP	Next Generation Application Protocol
NGMCS	Next Generation Mobile Communication System
NG-RAN	Next Generation Radio Access Network
NIST	National Institute for Standard and Technology
NRF	Network Repository Function
NWDAF	Network Data Analytics Function
OSS	Operations Support System
PDU	Protocol Data Unit
PEI	Permanent Equipment Identifier
PLMN	Public Land Mobile Network
PNG	Portable Network Graphics
QFI	QoS Flow Identifier
QoE	Quality of Experience
QoS	Quality of Service
QUIC	Quick UDP Internet Connections
RAN	Radio Access Network
REST	Representational State Transfer
RNN	Recurrent Neural Network
RPC	Remote Procedure Call
RRC	Radio Resource Control
SBI	Service-Based Interface
SLA	Service Level Agreement
SLS	Service Level Specification
SMF	Session Management Function
SRF	Signalling Routing Function

S-TMSI	S-Temporary Mobile Subscriber Identity
SUCI	Subscription Concealed Identifier
SUPI	Subscription Permanent Identifier
TAC	Tracking Area Code
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TMSI	Temporary Mobile Station Identifier
UDP	User Datagram Protocol
UE	User Equipment
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
UTF	Unicode Transformation Format
UUID	Universally Unique Identifier
W3	World Wide Web
W3C®	World Wide Web Consortium
WGS	World Geodetic System

4 Background and Design Rationale

4.1 Motivation

Next generation mobile networks are expected to support both baseline connectivity and beyond-connectivity services, including AI, sensing, computing, and immersive communication services [i.1], [i.2]. This requires the flexible composition of various network capabilities. Introducing AI agents into the mobile network is a promising approach to achieving this flexibility, given their capabilities for intent understanding, task decomposition, and autonomous execution capabilities. A set of AI agent use cases is described in [i.2], in which Network AI agents provide services directly to consumers. Associated network requirements include support for multi-agent operation, secure inter-agent communication, and intent-based interaction between customers and Network AI Agents.

In addition to the agents themselves, an agent-based mobile network requires common supporting components. For example, as described in [i.3], a tool repository is needed to store the common tools invocable by agents and a shared memory for maintaining the data and knowledge accessible to agents. In addition, [i.3] explains the use of a sandbox for testing and verifying the outputs of agents.

As described in [i.4], an Agent Repository Function (ARF) is needed to store and maintain per-agent information; this enables an agent to discover other agents. Network AI Agents therefore require interfaces to communicate with these common supporting components. Furthermore, agents in the core network are required to interact with entities outside the core network, such as UEs, Radio Access Networks (RANs), and cloud platforms to perform cross-domain collaboration. The interface and protocol design addressed in the present document covers this full range of interactions.

4.2 Gap Analysis with Respect to Existing Protocols

The lack of standardized interfaces for AI Agent interaction in mobile networks creates interoperability barriers, making it difficult for different AI components from various vendors to collaborate effectively. This fragmentation impedes end-to-end network automation and intelligent decision-making.

Two existing protocols are relevant as starting points. The Agent-to-Agent (A2A) protocol [i.7] defines methods and protocol data units for the task-oriented operations, such as task initiation, update, and cancellation. The Model Context Protocol (MCP) [i.8] enables AI agents and Large Language Models (LLMs) to communicate with external tools and data sources. This can be in the form of invoking a tool (also called a tool call) and data retrieval operations. Other examples of multi-agent communication protocols exist as well.

Unfortunately, neither protocol was designed with telecommunications use cases in mind. In a mobile communication system, Network AI Agents handle domain-specific tasks, including network control, session management, mobility management, and policy control. Network AI Agents exchange network-specific information including network state information, resource usage status, and Quality of Service (QoS) requirements. These requirements are not addressed by A2A or MCP as currently specified.

Additional gaps arise from the heterogeneous data and access control requirements of mobile networks. Network operator data, knowledge, and log data each carry distinct security requirements, necessitating differentiated authentication and authorization mechanisms and separate data operation methods or endpoints. Tool invocation by Network AI Agents similarly requires domain-specific access policies that generic tool-call mechanisms do not provide.

Table 4.2-1 summarizes the identified gaps and the corresponding interface types specified in clause 5.

Table 4.2-1: Interface types specified in the present document

Gap	Affected Interaction	Interface specified in clause 5
No telecom-specific task methods in A2A	Agent-to-agent	5.5
No intent-based UE-to-network interface	UE to Network AI Agent	5.2
No intent-based application-to-network interface	AF to Network AI Agent	5.3
No agent discovery/registration interface	Network AI Agent to ARF	5.4
No telecom-specific tool/data access policies in MCP	Network AI Agent to resources	5.6
No agent interface to RAN or compute infrastructure	Network AI Agent to infrastructure	5.7

Existing protocols such as A2A and MCP can serve as a basis for the agent-to-agent and agent-to-resource interfaces, but require extension to address the telecom-specific requirements identified above. The remaining interface types (i.e. to UEs, application functions, ARF, and infrastructure) have no existing protocol baseline and are specified entirely within the present document.

4.3 Goals and Design Principles

The primary goal of the present document is to specify how a Network AI Agent within a mobile communication system interacts with other network entities, UEs, and third-party applications in a secure and seamless fashion, irrespective of internal operation and implementation of each communication partner. For interactions with external communication partners, such as UEs and third-party application functions, the interface design enables service requests to be expressed as human-readable text (i.e. intents). These interfaces allow Network AI Agents to discover available tools and data sources, perform sub-tasks, and configure UE or RAN capabilities as needed to fulfil service requests. To support multi-agent collaboration, Network AI Agents should be able to discover other agents (including their agent information and other relevant metadata) via the ARF.

The interface design follows the principles listed below:

- **Encapsulation:** Network AI Agents can communicate with peer entities without exposing their internal implementation or reasoning logic [i.9], [i.10].
- **Traceability:** Actions taken by Network AI Agents and exchanged information are traceable and auditable, supporting accountability and regulatory compliance [i.11], [i.12].
- **Runtime Discoverability:** A Network AI Agent is able to determine how to communicate with a newly registered agent or tool by interrogating its published interface description, without requiring out-of-band configuration [i.14].
- **Forward and backward compatibility:** Interfaces and protocols accommodate new requirements without requiring redesign, and remain capable of processing messages from prior versions after they are upgraded [i.15], [i.16].
- **Security:** Interfaces support authentication, authorization, and access control appropriate to the sensitivity of the exchanged information, consistent with the heterogeneous security requirements of mobile network data and tools [5], [i.17], [i.18].

4.4 Relationship to the ENI System Architecture

The present document specifies communication interfaces within an agent-based mobile network. In the context of the ENI System Architecture defined in ETSI GS ENI 005 [4], the agent-based mobile network, which comprises Network AI Agents, the Agent Repository Function, and the infrastructure elements they interact with, functions as a **Class 3 Assisted System** as defined by ETSI GS ENI 005 [4]. A Class 3 Assisted System is a networked system with AI-based decision-making capabilities embedded in its internal control loops. Network AI Agents implement closed-loop, AI-based decision-making within the mobile network's internal control loops, satisfying the definition of a Class 3 Assisted System as specified in clause 4.4.2.5 of ETSI GS ENI 005 [4].

An alternative mapping in which the agent-based mobile network constitutes an ENI System rather than an Assisted System was considered but not adopted in the present document. This mapping is architecturally invalid for two reasons. First, an ENI System is defined in ETSI GS ENI 005 [4] as a cognitive system comprising a specific set of modular Functional Blocks (Data Ingestion, Normalization, Knowledge Management, Context-Aware Management, Cognition Management, Situational Awareness, Model Driven Engineering, Policy Management, Denormalization, and Output Generation). The agent-based mobile network specified in the present document does not implement these Functional Blocks and therefore does not satisfy the definition of an ENI System. Second, even if that criterion were met, the mapping would require the ENI System to be embedded within the Assisted System. This configuration is explicitly prohibited by clause 6.2.3.1 and Table 4-1 of ETSI GS ENI 005 [4].

As specified in ETSI GS ENI 005 [4], the ENI System operates externally to the Assisted System and communicates with it via ENI External Reference Points (ERPs), mediated by an API Broker. Specifically, the present document shall be used in conjunction with the ENI External Reference Points defined in ETSI GS ENI 005 [4], which govern the observation, management, and adaptation of the agent-based Assisted System by an ENI System. The relevant ERPs fall into two categories:

1) OSS-facing ERPs, which are most applicable to the agent population as a managed system:
   - $E_{oss\text{-}eni\text{-}dat}$;
   - $E_{oss\text{-}eni\text{-}cmd}$; and
   - $E_{oss\text{-}eni\text{-}pol}$;
2) Infrastructure ERPs, which are relevant for the RAN/compute interactions:
   - $E_{inf\text{-}eni\text{-}dat}$; and
   - $E_{inf\text{-}eni\text{-}cmd}$.

The interfaces specified in the present document are internal to the Assisted System boundary as defined in ETSI GS ENI 005 [4]. They specify how Network AI Agents interact among themselves and with the UEs, tools, data sources, and infrastructure elements that constitute the agent-based Assisted System.

The ENI System has the ability to observe, advise, and manage the agent-based mobile network (as a whole or in part) via ERPs as defined in clause 7 of ETSI GS ENI 005 [4]. Such management includes, but is not limited to, ingesting telemetry from the agent population through data-plane ERPs; issuing policy-based recommendations or management commands to the ARF or individual agents through command-plane ERPs; and adapting agent deployment or configuration through the ENI closed-loop control architecture defined in clause 6.3.1.5 of ETSI GS ENI 005 [4].

The present document does not specify ENI External Reference Points or the ENI API Broker. The specification of ENI External Reference Points for the agent-based Assisted System is deferred to a future work item; see clause 7.2.

5 Interface Design

5.1 Definitions

A Network AI Agent within a mobile communication system communicates with six peer entity categories, each corresponding to a distinct interface type. Figure 5.1-1 depicts these six interface types. In the figure, each line represents a logical interface between a Network AI Agent and a peer entity; the interface is bidirectional unless otherwise specified in Figure 5.1-1.

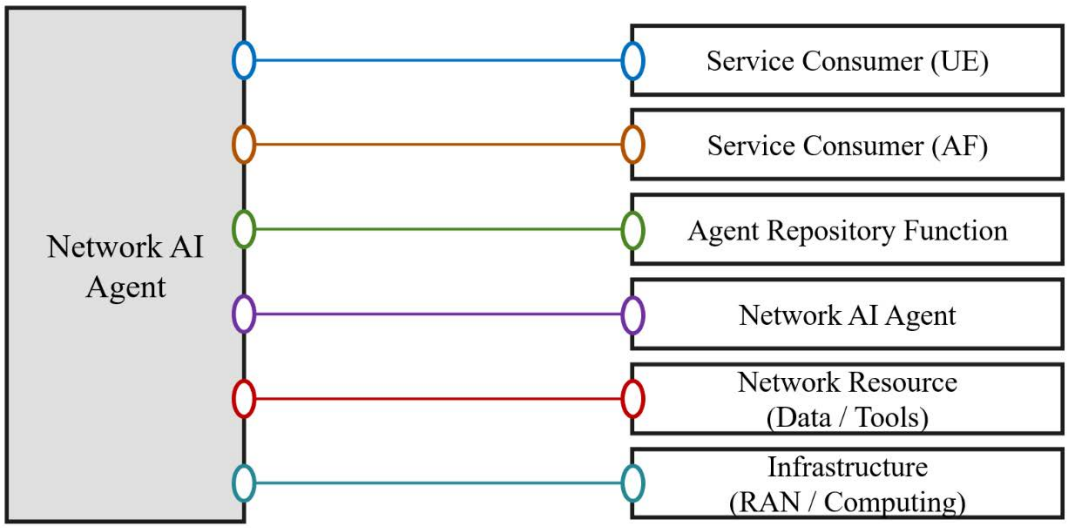


Figure 5.1-1: Six types of interfaces of a Network AI Agent

The peer entity categories are defined as follows:

- Service Consumer is an entity that initiates a service request to a Network AI Agent. A Service Consumer may be a UE or a third-party Application Function (AF). When the Service Consumer is a UE, service requests are conveyed via an enhanced Non-Access Stratum (NAS) protocol as described in clause 5.2. When the Service Consumer is an AF, service requests are conveyed via the interface described in clause 5.3.
- Agent Repository Function, as defined in clause 3.1, is a network function responsible for storing and maintaining lifecycle information about registered AI Agents. Network AI Agents interact with the ARF for agent registration, update, deregistration, and discovery, as specified in clause 5.4.
- Network Resource includes the data and tools accessible to Network AI Agents within the mobile network. Data resources include operator policy data, user subscription data, network knowledge bases, and log data. Tool resources are callable functions exposed to Network AI Agents for execution. The interfaces to data and tool resources are specified in clauses 5.6.1 and 5.6.2 respectively.
- Infrastructure comprises the physical and logical network infrastructure elements, including the RAN nodes and computing platform, whose capabilities can be configured or invoked by Network AI Agents. Infrastructure elements expose callable interfaces to Network AI Agents. The interface design follows the same method structure as the tool resource interface specified in clause 5.6.2, with extensions for RAN- and compute-specific operations as specified in clause 5.7.

NOTE: The term "Network AI Agent" appearing on both sides of the agent-to-agent interface in Figure 5.1-1 reflects the symmetric nature of that interface; either agent acts as the initiator. See clause 5.5.

5.2 Interface between Network AI Agent and UE

5.2.1 Requirements

Next-generation mobile networks are expected to deliver AI, sensing, and data services to UEs in addition to baseline connectivity services [i.1]. For resource-constrained intelligent edge devices that have limited processing capabilities and resources, such as smartphones and mobile robots, Network AI Agents shall provide services that assist in handling complex, open-ended tasks that cannot be expressed as fixed procedural requests. Service requests originating from UEs are likely to be intent-based, variable in content, and non-deterministic.

The interface between a UE and a Network AI Agent shall allow UE-originated service interactions. Specifically, this interface:

- shall allow a UE to initiate a new service;
- shall allow a UE to update an existing service;

- shall allow a UE to retrieve service information of an existing service; and
- shall allow a UE to delete an existing service. In addition to UE-originated interactions, the interface shall support Network AI Agent-originated interactions in which the Network AI Agent queries or configures capabilities resident on the UE.

UEs may expose all or some of their capabilities to Network AI Agents to assist them in service delivery to other UEs or applications function. Such capability configuration shall only be performed following explicit UE authorization or prior capability exposure consent established during service initiation.

Therefore, the interface between a Network AI Agent and UE:

- shall allow Network AI Agents to query the available UE capabilities, and
- shall allow Network AI Agents to configure one or more specific UE capabilities.

For the purposes of the present document, UE capability refers to the set of functions that a UE supports and can expose to the network, including:

- radio capabilities (e.g. supported frequency bands, number of antennas, modulation schemes);
- sensing capability (e.g. supported sensing schemes, sensing area);
- AI inference capabilities (e.g. supported model types, available compute resources, inference latency bounds; and
- data processing capability (e.g. supported data types, data processing schemes).

Where an UE hosts an UE AI Agent as defined in clause 3.1, the interactions described in this clause may be mediated by the UE AI Agent rather than the UE platform directly.

NOTE 1: The protocol behaviour is identical in both cases; the distinction is internal to the UE implementation and outside the scope of the present document.

NOTE 2: ETSI GS ENI 019 [8] defines the ENI Extended Core (information) Model. Extensions to ETSI GS ENI 019 [8] to better support the present document are For Further Study (FFS).

5.2.2 Methods and Data Objects

Table 5.2.2-1 specifies the methods that the Network AI Agent-to-UE interface shall support based on the requirements in clause 5.2.1. Protocol encoding of these methods is specified in clause 5.2.3.

Table 5.2.2-1: Supported methods for interface exposed by Network AI Agents to UEs

Method Name	Description	Inputs	Outputs
service/create	Initiate a new service	ServiceConfig as shown in Table 5.2.2-2	OperationResult as defined in Table 5.2.2-4.
service/update	Update an existing service. Shall fail if no service with the specified Service ID exists.	ServiceConfig as shown in Table 5.2.2-2	OperationResult as defined in Table 5.2.2-4.
service/get	Query the information of a service	serviceIDContent : string[1..1] and serviceIDFormat : string[1..1]	Fields as shown in Table 5.2.2-5.
service/delete	Delete an existing service, along with its context and resources	serviceIDContent : string[1..1] and serviceIDFormat : string[1..1]	OperationResult as defined in Table 5.2.2-4.

When the UE requests to initiate a service using the service/create method, the request shall contain a service ID, which comprises serviceIDContent and serviceIDFormat and is assigned by the UE, the identifier of the UE that consists of ueIDContent and ueIDFormat, and a message identifier used to correlate the response to this request. The request shall also carry userIntentContent and userIntentFormat fields that together constitute a natural language description of the requested service. The combination of UE ID and Service ID shall be unique within the scope of a Network AI Agent. Optionally, the UE may specify one or more accepted response modalities that the Network AI Agent can use in its response (e.g. text, image). Table 5.2.2-2 specifies the fields of the ServiceConfig object that is provided as input to a "service/create" request when the UE requests to update an existing service:

Table 5.2.2-2: Fields of the ServiceConfig data type

Field Name	Type	Presence	Description
serviceIDContent	string[1..1]	M	Value of the serviceID identifier. This forms the composite serviceID field together with the serviceIDFormat field. A composite key is necessary because there is no standard format applicable for all services. This field shall not be NULL or empty.
serviceIDFormat	string[1..1]	M	The format or namespace of the serviceIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite serviceID field together with the serviceIDContent field. This field shall not be NULL or empty.
schemaVersion	string[1..1]	M	Version of the serviceConfig schema, which conforms to SemVer 2.0.0 (see [6]). This enables schema evolution across deployments.
ueIDContent	string[1..1]	M	The actual value of UE identifier, which may have multiple formats. Depending on context, a UE may be identified by: • SUPI (Subscription Permanent Identifier) - A globally unique permanent identifier assigned to each subscriber on a mobile network. Either an International Mobile Subscriber Identity (IMSI) or a Network Access Identifier (NAI) [1]; • SUCI (Subscription Concealed Identifier) - the privacy-protected form of SUPI used over the air; • GUTI (Globally Unique Temporary Identifier) - the temporary identity assigned by the NF in the core network, changes on re-registration; • PEI (Permanent Equipment Identifier) - International Mobile Equipment Identity (IMEI)-based, identifies the device not the subscription; • TMSI (Temporary Mobile Subscriber Identity) - short form of GUTI used in RRC/NAS. This field shall be consistent with the UE identity established during the NAS session in which this request is carried.
ueIDFormat	string[1..1]	M	The format or namespace of the ueIDContent field (e.g. SUPI, SUCI, GUTI, PEI, 5G-S-TMSI [9]. This forms the composite ueID field together with the ueIDContent field. This field shall not be NULL or empty. This field shall be consistent with the UE identity established during the NAS session in which this request is carried.
messageID	string[1..1]	M	Identifier of the message to be used in the response message. It is a per-request correlation identifier and the Network AI Agent shall echo this value unchanged in the corresponding response message.
userIntentContent	string[1..1]	M	Value of the intent expression. This forms a composite key together with the userIntentFormat field. A composite key is necessary because there is no standard format applicable for all types of user intents. This field shall not be NULL or empty.
userIntentFormat	string[1..1]	M	The format or namespace of the userIntentContent field (e.g. natural-language-en, gsma-rcs-intent-1.0). This forms a composite key together with the userIntentContent field. This field shall not be NULL or empty.
modalitySet	ModalityInfo[0..*]	O	Data modalities that are accepted by UE. The object ModalityInfo is defined in Table 5.2.2-3.
NOTE: A presence value of "M", "O", and "CM" represent "mandatory", "optional", and "conditional mandatory", respectively.			

Table 5.2.2-3 specifies the fields in the ModalityInfo data type.

Table 5.2.2-3: Fields of the ModalityInfo data type

Field Name	Type	Presence	Description
modalityType	enum[1..1]	M	The type of data modality, including text, image, audio, video, structured data.
modalityFormat	string[0..1]	O	Encoding format of data, such as UTF-8, JPEG, PNG, MP4, JSON, etc.
modalityVersion	string[0..1]	O	Schema version for data modality.

After the Network AI Agent receives the service/create or service/update request, it will return a response message to the UE, which contains the information shown in the following Table 5.2.2-4.

Table 5.2.2-4: Fields of the OperationResult data type

Field Name	Type	Presence	Description
status	enum[1..1]	M	The status can either be "success" or "failure".
errorCode	string[1..1]	CM	Machine-readable error code. Present only when status is "failure".
errorMessage	string[1..1]	CM	Human-readable error description. Present only when status is "failure". If the status is "failure", then the entry shall NOT be NULL or empty.

When a UE queries the service information using the service/get method that contains the serviceIDContent and serviceIDFormat, then the Network AI Agent shall return the current service status, a Service Context ID associated with the specific service context and, where applicable, service results or error information, as specified in Table 5.2.2-3.

If the service request is accepted, the Network AI Agent shall create the corresponding service context object containing all the information relevant to the service. More specifically:

1) the user intent;
2) service requirements;
3) service constraints;
4) Network AI Agents involved in the service;
5) a list of tasks allocated to each involved Network AI Agent;
6) the chat history between the Network AI Agent and the UE; and
7) the chat history between Network AI Agents.

The service context information shall be assigned a Service Context ID for subsequent retrieval. The service context may be large in size. Therefore, it may be stored at a shared node accessible to all Network AI Agents participating in the service; in this case, the Service Context ID functions as a retrieval key. The Network AI Agent shall return the Service Context ID to the UE in response to a service/get request.

Service context contains UE identity and interaction history. Retention, access control, and processing of service context data shall comply with applicable data protection and privacy requirements of the operator and applicable regulations.

NOTE: Service context storage and access among Network AI Agents is an implementation matter outside the normative scope of the present document. The present document specifies only the Service Context ID field returned to the UE.

If the service status is failed, the response shall include service error information as specified in Table 5.2.2-5. The response may also contain a service result.

Table 5.2.2-5: Fields of the outputs for the service/get method

Field Name	Type	Presence	Description
serviceIDContent	string[1..1]	M	Value of the serviced identifier requested by UE.
serviceIDFormat	string[1..1]	M	Format or namespace of the serviceIDContent field.
messageID	string[1..1]	M	Identifier of the corresponding request message.
timestamp	string[1..1]	M	Time at the service information retrieval. It shall be formatted in accordance with ISO 8601 [14] (e.g. "2026-05-14T17:00:00Z").
serviceStatus	ServiceStatus[1..1]	M	Current status of the service with serviceID. See the ServiceStatus enumeration in Table 5.2.2-6. The state machine governing transitions between these states is shown in Figure 5.2.2-1.
serviceContextID	string[0..1]	O	Identifier of the service context associated with this service. This identifier can be used to correlate related service and task invocations across interfaces. The serviceContextID is present only when a service context has been created for the service.
serviceContext	ContextInfo[0..1]	O	Optional context associated with this service, such as UE session attributes, UE application state, or slice information. When present, the serviceContext shall be encoded as a ContextInfo object as defined in Table 5.2.2-5a.
serviceResultSet	ServiceResult[0..1]	O	Immediate or final results for the service. The fields contained in the ServiceResult are described in Table 5.2.2-7.
serviceErrorInfo	string[1..1]	CM	Description of the cause for service failure. This is required only when the service status is failed.
NOTE 1: The service context means all the information relevant to the service, including the UE intent, the Network AI Agents associated with the service, and previous message exchange between the UE and the Network AI Agents. NOTE 2: Business rules, SLAs and SLSs, and other relevant information is also typically part of context. See ETSI GS ENI 005 [4] and ETSI GS ENI 019 [8] for additional information.			

Table 5.2.2-5a defines the values of the ContextInfo data type.

Table 5.2.2-5a: Fields of the ContextInfo data type

Field Name	Type	Presence	Description
contextType	string[1..1]	M	Identifier of the context type (e.g. "ue-session", "af-session", "ran-cell", "slice", "agent-task").
contextSchemaID	URI[0..1]	M	URI identifying a JSON schema, ontology, or information model that defines the structure of contextData, if applicable. A URI is a mandatory string as defined in IETF RFC 3986 [7].
contextData	object[1..1]	M	JSON object containing context attributes. Where contextSchemaID is present, the structure of contextData shall conform to the referenced schema. Otherwise, the structure is implementation-specific and agreed out-of-band between communicating entities.
NOTE: The ContextInfo and related data types are intentionally aligned with the context modelling principles defined in ETSI GS ENI 019 [8]. The detailed representation and aggregation patterns of ContextInfo objects within an ENI System are specified in ETSI GS ENI 019 [8]; the present document only defines how context information is attached to the interfaces of the agent-based mobile network.			

Table 5.2.2-6 defines the values of the ServiceStatus data type.

Table 5.2.2-6: Enumerated value of ServiceStatus data type

Value	Description
Pending	The service request has been received, but processing has not started yet; the next state is Active.
Active	The service is being processed.
Completed	The service is successfully fulfilled.
Partially-completed	The service completed, but only with partial results.
Cancelled	The service was cancelled by the UE or Network AI Agent.
Failed	The service was terminated due to error.
Timeout	The service was terminated due to time expiry.
NOTE: The states Completed, Partially-completed, Cancelled, Timeout, and Failed are terminal states. No transitions are allowed to emanate from any terminal state to any other state.	

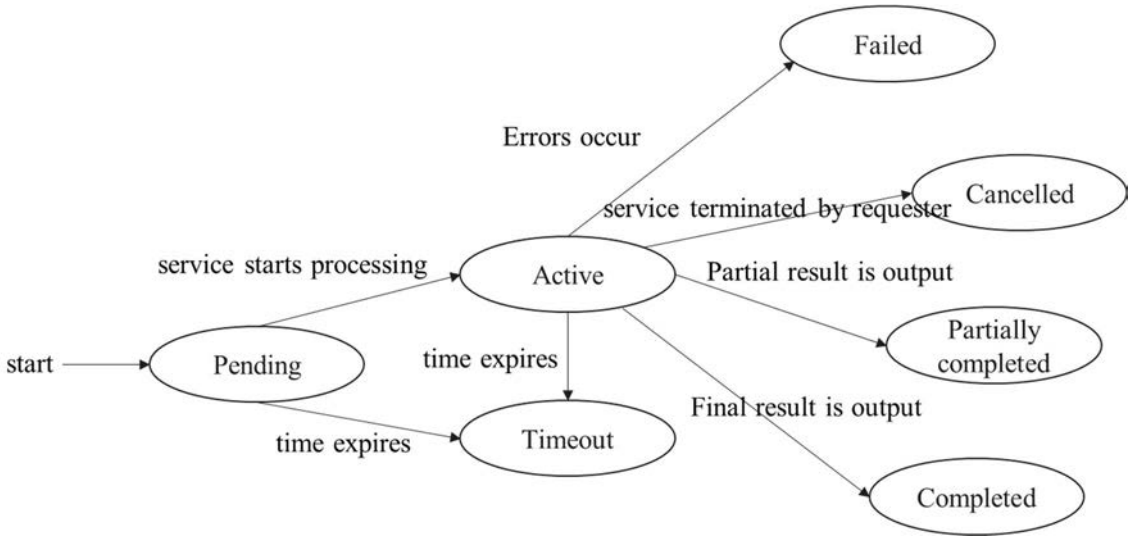


Figure 5.2.2-1: State machine transition diagram

Table 5.2.2-7 defines the fields of the ServiceResult data type.

Table 5.2.2-7: Fields of the ServiceResult data type

Field Name	Type	Presence	Description
modalitySet	ModalityInfo[1..1]	M	Data modality of the service result. The ModalityInfo object is defined in Table 5.2.2-3. The modalityType and modalityFormat fields together determine the encoding of the serviceResultContent field.
serviceResultContent	string[1..1]	M	Value of the service result, encoded as specified by the modalitySet field. For modalityType = "text", the value shall be a UTF-8 string. For modalityType = "image", the value shall be a Base64-encoded byte string. For modalityType = "audio" or "video", the value shall be a URI referencing a network-accessible resource. For modalityType = "structured data", the value shall be a JSON-encoded object conforming to the schema identified by modalityFormat.

A UE may have a camera or sensors to collect sensing data. UEs may expose local capabilities, including sensing, AI inference, and data processing, to Network AI Agents upon request. When the UE reports its capabilities, it shall indicate the capabilityID and the set of configurable parameters associated with that capability. Table 5.2.2-8 specifies the methods exposed by UEs to Network AI Agents for capability discovery and configuration. When a Network AI Agent queries the available capabilities on an UE, the UE shall return, for each available capability, the UE ID, the capabilityID, and the configurable parameters for the capability, as specified in Table 5.2.2-5.

Table 5.2.2-8: Supported methods for interface exposed by UEs to Network AI Agents

Method Name	Description	Inputs	Outputs
ueCapability/list	Acquire the set of capabilities available on a specific UE	NA	UECapabilitySet as defined in Table 5.2.2-9; one entry per supported capability.
ueCapability/configure	Configure the parameters of a specific UE capability. Shall only be invoked following UE consent as established during service initiation (see clause 5.2.1). Does not activate or execute the capability	capabilityID: string[1..1], defined in Table 5.2.2-9 capabilityParams: CapabilityParams[1..1] (type-specific parameters per Table 5.2.2-9)	ConfigAckResult: configuration status, and confirmation of accepted parameters on success. In case of failure, error code and error message are included. The fields are defined in Table 5.2.2-14.
ueCapability/activate	Activate a previously configured UE capability. The capability shall have been configured via ueCapability/configure prior to activation	capabilityID: string[1..1]	ActivationResult: activation status and execution output on success; on failure, error code and message. The fields are defined in Table 5.2.2-15.
ueCapability/deactivate	Deactivate a previously activated UE capability	capabilityID: an identifier of the UE capability	DeactivationResult: deactivation status on success; on failure error code and error message are included. The fields are defined in Table 5.2.2-15.

Table 5.2.2-9 defines the fields of the UECapabilitySet data type.

Table 5.2.2-9: Fields of the UECapabilitySet data type

Field Name	Type	Presence	Description
ueIDContent	string[1..1]	M	The UE identifier (see definition in Table 5.2.2-2). Hence, ueIDContent is the actual identifier value, and ueIDFormat defines which identification is used.
ueIDFormat	string[1..1]	M	The format or namespace of the ueIDContent field (e.g. SUPI, SUCI, GUTI, PEI, 5G-S-TMSI [9]). This forms the composite ueID field together with the ueIDContent field. This field shall not be NULL or empty. This field shall be immutable after the Network AI Agent is registered.
capabilityID	string[1..1]	M	An identifier of the UE capability, e.g. a capability name.
parameterSet	string[1..1]	M	Format description of configurable parameters for the UE capability, e.g. JSON.
capabilityType	enum[1..1]	M	Type discriminator identifying the capability category and thus the applicable parameter schema. Allowed values are "radio", "sensing", "ai-inference", "data-processing". The value of this field determines which fields in capabilityParameters are applicable. (see Table 5.2.2-10 through Table 5.2.2-13 for per-type definitions).
capabilitySchemaVer	string[1..1]	M	The schema version for the capabilityParameters structure, expressed as a SemVer 2.0.0 string (major.minor.patch). This field enables forward- and backward-compatible schema evolution per capability type. The value shall not be null or empty.
capabilityParams	CapabilityParamInfo[1..*]	M	The parameter set for this capability. The applicable fields and their types are determined by the value of capabilityType and capabilitySchemaVer. See Table 5.2.2-10 through Table 5.2.2-13 for per-type parameter definitions.

Table 5.2.2-10 defines the fields of the CapabilityParamInfo data type when the capabilityType field is "radio".

Table 5.2.2-10: Fields of the CapabilityParamInfo data type when the capabilityType field is "radio"

Parameter Name	Type	Presence	Value
freqBandsSupported	enum[1..1]	M	Supported frequency bands. Values include "n1", "n3", "n5", "n7", "n8", "n20", "n28", "n257", "n258". At least one value shall be present.
numAntennas	enum[1..]	M	Supported number of antennas. Values include "1T2R", "1T4R", "2T2R", "2T4R". At least one value shall be present.
modulationSchemes	enum[1..]	M	Supported modulation schemes. Values include "QPSK", "16QAM", "64QAM", "256QAM", "1024QAM". At least one value shall be present.

Table 5.2.2-11 defines the fields of the CapabilityParamInfo data type when the capabilityType field is "sensing".

Table 5.2.2-11: Fields of the CapabilityParamInfo data type when the capabilityType field is "sensing"

Parameter Name	Type	Presence	Value
sensingSchemesSupp	enum[1..1]	M	Supported sensing schemes. Values include "passive", "active", "cooperative". At least one value shall be present.
sensingArea	SensingArea[1..1]	M	Geographic sensing area within which this RAN node can perform sensing for this capability, represented as a SensingArea object as defined in Table 5.2.2-11a.
sensingDuration	TimeInterval[0..1]	O	Time interval during which this sensing capability is available or configured, represented as a TimeInterval object as defined in Table 5.2.2-11b. If absent, the capability is considered continuously available subject to operator policy.
sensingAccuracy	number[1..1]	M	Accuracy of the sensing capability, expressed in metres. This value shall be a non-negative real number.

Table 5.2.2-11a defines the SensingArea data type. This abstracts the concept of a "sensing area" to be independent of what entity is using it (e.g. a UE or the RAN infrastructure) or what that concept applies to.

Table 5.2.2-11a: Fields of the SensingArea data type

Parameter Name	Type	Presence	Value
startLongitude	number[1..1]	M	Western longitude boundary of the sensing area, expressed in decimal degrees using WGS-84. Allowed range is -180,0 to +180,0. The value is inclusive.
endLongitude	number[1..1]	M	Eastern longitude boundary of the sensing area, expressed in decimal degrees using WGS-84. Allowed range is -180,0 to +180,0. The value is inclusive.
startLatitude	number[1..1]	M	Suthern longitude boundary of the sensing area, expressed in decimal degrees using WGS-84. Allowed range is -90,0 to +90,0. The value is inclusive.
endLatitude	number[1..1]	M	Northern longitude boundary of the sensing area, expressed in decimal degrees using WGS-84. Allowed range is -90,0 to +90,0. The value is inclusive.

The following constraints apply: values shall satisfy startLongitude $\leq$ endLongitude and startLatitude $\leq$ endLatitude. Representation of sensing areas that cross the 180-degree meridian is for further study.

Table 5.2.2-11b defines the TimeInterval data type. This abstracts the concept of a time duration to be independent of what entity is using it (e.g. a UE or the RAN infrastructure) or what that concept applies to.

Table 5.2.2-11b: Fields of the TimeInterval data type

Parameter Name	Type	Presence	Value
startTime	string[1..1]	M	Start of the interval, expressed as a UTC timestamp in ISO 8601 [14] format (e.g. "2026-06-08T12:00:00Z").
endTime	string[1..1]	M	End of the interval, expressed as a UTC timestamp in ISO 8601 [14] format (e.g. "2026-06-08T12:00:00Z"). The value of endTime shall be later than or equal to startTime.
NOTE: The interval [startTime, endTime] is inclusive at both endpoints. Representation of recurring or non-contiguous time intervals is for further study.			

Table 5.2.2-12 defines the CapabilityParamInfo data type when the capabilityType enum is "ai-inference".

Table 5.2.2-12: Fields of the CapabilityParamInfo data type when capabilityType is "ai-inference"

Parameter Name	Type	Presence	Value
modelTypesSupp	enum[1..1]	M	Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Generative Adversarial Network (GAN), Large Language Model (LLM).
compResAvail	integer[1..1]	M	An integer value, the unit is Floating-point Operations Per Second (FLOPS).
compResReq	integer[1..1]	M	An integer value defining the computing resources required for a particular set of tasks. The unit is Floating-point Operations Per Second (FLOPS).
inferenceLatency	float[1..1]	M	A float value, the unit is millisecond.

Table 5.2.2-13 defines the CapabilityParamInfo data type when the capabilityType field is "data-processing".

Table 5.2.2-13: Fields of the CapabilityParamInfo data type when the capabilityType field is "data-processing"

Parameter Name	Type	Presence	Value
suppDataTypes	enum[1..1]	M	text, image, audio, video, structured data, vector.
dataProcSchemes	enum[1..1]	M	data feature extraction, data fusion, data compression, data cleaning, data transformation, data encryption.

Table 5.2.2-14 defines the ConfigAckResult data type.

Table 5.2.2-14: Fields of the ConfigAckResult data type

Field Name	Type	Presence	Description
status	enum[1..1]	M	Enumerated value of the UE capability configuration result. Valid values are "success", "partial", and "failure". "Partial indicates that some parameters were accepted and others rejected (see note).
acceptedParams	CapabilityParamInfo[1..1]	CM	Part or all of the parameter values that are accepted and applied by the UE (see Table 5.2.2-10 through Table 5.2.2-13). Present only when the status is "success" or "partial".
rejectedParams	CapabilityParamInfo[1..1] [1..*]	CM	Part or all of the parameters that were rejected, each with a reason code. Present when status is "partial" or "failure".
errorCode	string[1..1]	CM	Present only when status is "failure".
errorMessage	string[1..1]	CM	Present only when status is "failure".
NOTE: A UE accepts some configuration parameters and constrains or rejects others (e.g. it supports the requested sensing area but not the requested resolution). Without 'partial', the Network AI Agent lacks visibility into the subset of configuration applied, which affects subsequent ueCapability and activates correctness.			

Table 5.2.2-15: Fields of the ActivationResult data type

Field Name	Type	Multiplicity	Description
status	enum[1..1]	M	Enumerated value of the UE capability activation result, it is either "success" or "failure".
executionOutput	string[0..1]	CM	Capability execution result payload. Present when status is "success". Format is capability-type-dependent - see note.
errorCode	string[0..1]	CM	Value of error code. Present only when status is "failure".
errorMessage	string[0..1]	CM	Descriptive error information. Present only when status is "failure".
NOTE: The format of executionOutput is determined by the capabilityType of the activated capability (as defined in Table 5.2.2-9) and the capabilitySchemaVersion. Receivers use the capabilityType discriminator to select the appropriate output parser.			

5.2.3 Protocol

In 5G, the NAS protocol is used to transfer control signalling between the UE and the core network. A variety of payload container types are defined within NAS layer [2]. All NAS messages are initially received by the Access and Mobility Management Function (AMF), which is responsible for processing NAS messages addressed to it and for transparently routing other NAS messages to the appropriate network function (e.g. to the SMF for session management signalling).

In order to allow in-network processing of UE-initiated intent-based service requests by Network AI Agents, the UE shall transmit the user intent via an enhanced NAS layer to the Network AI Agent. This requires an extension of the NAS protocol in which a new NAS payload container type is defined to carry user intent between the UE and the core network. The detailed specification of this NAS extension, including the message type indicator, container encoding, and protocol procedure, is outside the scope of the present document and is deferred to the relevant 3GPP specifications. The present document specifies the requirements that the NAS extension shall satisfy in order to convey the methods and data objects defined in clause 5.2.2.

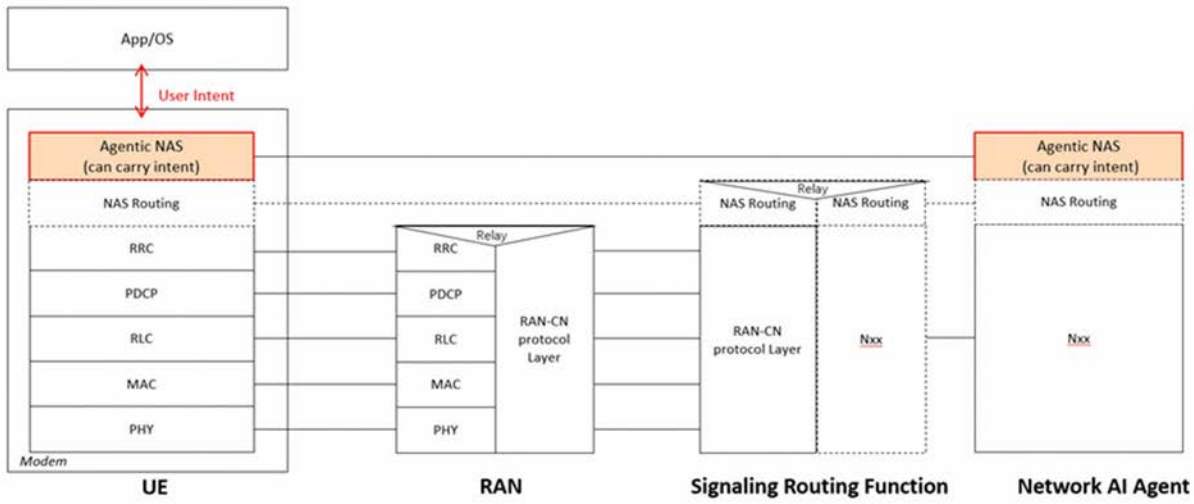


Figure 5.2.3-1: Intent-based UE-Network AI Agent communication via NAS protocol

Figure 5.2.3-1 illustrates a candidate end-to-end protocol stack for intent transmission between the UE and a Network AI Agent. A new network function, called the Signalling Routing Function (SRF), is shown as the entity responsible for routing intent-based NAS messages to the appropriate Network AI Agent. NAS terminates at the Network AI Agent, which decapsulates the intent payload from the NAS container and takes actions to fulfil intent.

NOTE 1: The Signalling Routing Function (SRF) is a candidate network function under study in 3GPP. Its introduction into the 6G system architecture has not been standardized as of the date of publication of the present document.

NOTE 2: The interface between the SRF and the Network AI Agent, labelled Nxx in Figure 5.2.3-1, is a placeholder reference point pending 3GPP specification.

Upon receiving the intent payload, the Network AI Agent shall perform intent interpretation, dynamic task composition, and orchestration, as well as other necessary operations to effectively understand, process, and satisfy the UE's service request.

The method names shall be encoded as the "method" parameter of the JSON-RPC 2.0 request object (see [3]).

Multiple Network AI Agents may be deployed in the core network, with different Network AI Agents responsible for handling intent-based service requests corresponding to different network services. This enables specialized processing and improved scalability. For example, a dedicated Network AI Agent may be responsible for managing requests related to connectivity services such as connection establishment, mobility management, and session continuity, while a different Network AI Agent may be responsible for handling requests related to data processing and storage.

5.2.4 Security

5.2.4.1 Authentication

Authentication of User Equipment accessing the interface defined in clause 5.2 of the present document shall comply with clause 5.8.2.2 of the present document. A UE shall authenticate using the authentication procedure as defined in prior to establishing any service interaction with a Network AI Agent. A Network AI Agent shall reject any service request received from a UE that has not successfully completed authentication within the current NAS session.

NOTE: UE authentication for the interface defined in clause 5.2 is performed at the NAS layer as part of the registration procedure. Application-layer mTLS is not applicable to UE-originated requests conveyed via the enhanced NAS protocol defined in clause 5.2.3.

5.2.4.2 Authorization

Authorization of User Equipment shall comply with clause 5.8.3.1 of the present document. A UE shall obtain explicit operator authorization prior to invoking any method defined in clause 5.2.2 of the present document. Authorization shall be established during the NAS service initiation procedure defined in clause 5.2.3 and shall be bound to the authenticated UE identity (SUPI) for the duration of the service.

The following authorization requirements apply per method:

1) Service/create: requires operator authorization for service initiation scoped to the requesting UE's subscription profile. The Network AI Agent shall verify that the ueIDContent and ueIDFormat fields in ServiceConfig are consistent with the UE identity established during the NAS session in which the request is carried.

2) Service/update: requires operator authorization scoped to service update for the requesting UE, and shall only succeed if the service identified by the composite serviceID was previously created by the same UE.

3) Service/get: requires operator authorization scoped to service retrieval. The Network AI Agent shall only return information pertaining to services created by the requesting UE.

4) Service/delete: requires operator authorization scoped to service deletion, and shall only succeed if the service was created by the requesting UE.

5) UeCapability/configure, ueCapability/activate, and ueCapability/deactivate: require explicit UE consent in addition to operator authorization, as specified in clause 5.2.1 of the present document. A Network AI Agent shall not invoke ueCapability/configure, ueCapability/activate, or ueCapability/deactivate unless UE consent has been established and remains valid for the capability being configured or activated.

5.2.4.3 Data Access Constraints

A Network AI Agent shall not expose Class 3 or Class 4 data as defined in clause 5.8.3.5 of the present document to a UE via the interface defined in clause 5.2 of the present document. Service context information returned in response to service/get shall be filtered to exclude any network-internal data not pertaining to the requesting UE's own service context. Capability configuration parameters returned in response to ueCapability/list shall be limited to the capabilities of the requesting UE and shall not include configuration information pertaining to other UEs.

5.2.4.4 Transport Security

Transport security for the UE interface shall comply with clause 5.8.4 of the present document. Service requests conveyed via the enhanced NAS protocol defined in clause 5.2.3 are protected by the NAS security context established during 5G primary authentication, including NAS confidentiality and integrity protection as defined in 3GPP TS 24.501 [2] and 3GPP TS 33.501 [5]. Where the UE interface is realized over an application-layer protocol binding (see clause 5.2.3), all interactions shall additionally be carried over TLS 1.3 or later. TLS 1.2 fallback as permitted by clause 5.8.4 shall not be used for the application-layer UE interface. TLS 1.3 is mandatory for this interface due to the sensitivity of the data carried, including UE identity (ueIDContent) and user intent content.

5.3 Interface between Network AI Agent and Third-Party Applications

5.3.1 Requirements

Interacting with a Network AI Agent within the mobile network allows third-party applications to shift from rigid, request-response protocols to flexible, intent-based operations. This reduces the effort and complexity for the application developer by eliminating the need to understand complex network parameters and allows them to communicate a high-level outcome. Authorization of AFs shall be performed using the mechanisms defined in clause 5.8, consistent with the common API framework for 3GPP northbound APIs [13]. Thus, Network AI Agents:

- shall allow an AF to initiate a new service;
- shall allow an AF to update an existing service;
- shall allow an AF to retrieve service information of an existing service; and
- shall allow an AF to delete an existing service.

Moreover, the interface between the network and application should allow the Network AI Agents to discover and invoke tools that are hosted and exposed by the application. Therefore, the interface shall allow Network AI Agents to:

- discover and invoke the AF tools.

5.3.2 Methods and Data Objects

When the AFs as consumers to request Network AI Agents to provide services, the following methods should be supported.

Table 5.3.2-1: Supported methods for interface exposed by Network AI Agents to AFs

Method Name	Description	Inputs	Outputs
afService/create	Create a new service for an AF. Shall fail if a service with the specified serviceID composite key already exists.	AFServiceConfig as shown in Table 5.3.2-2	OperationResult as defined in Table 5.2.2-4.
afService/update	Update an existing service for an AF. Shall fail if no service with the specified Service ID exists.	AFServiceConfig as shown in Table 5.3.2-2	OperationResult as defined in Table 5.2.2-4.
afService/get	Query the information of a service previously created by an AF.	serviceIDContent : string[1..1] and serviceIDFormat : string[1..1]	Fields as shown in Table 5.2.2-5.
afService/delete	Delete an existing service for an AF, along with its context and resources.	serviceIDContent : string[1..1] and serviceIDFormat : string[1..1]	OperationResult as defined in Table 5.2.2-4.

When an AF initiates a new service or updates an existing service, the input data object AFServiceConfig shall contain the fields defined in Table 5.3.2-2. The AFServiceConfig structure is equivalent to ServiceConfig (Table 5.2.2-2), with the ueIDContent and ueIDFormat fields replaced by afIDContent and afIDFormat respectively.

Table 5.3.2-2: Fields of the AFServiceConfig data type

Field Name	Type	Presence	Description
serviceIDContent	string[1..1]	M	Value of the serviceID identifier. This forms the composite serviceID field together with the serviceIDFormat field. A composite key is necessary because there is no standard format applicable for all services. This field shall not be NULL or empty. See note 1.
serviceIDFormat	string[1..1]	M	The format or namespace of the serviceIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite serviceID field together with the serviceIDContent field. This field shall not be NULL or empty. See note 1.
afIDContent	string[1..1]	M	The value of the AF identifier. This is required because AF identifiers have no universal format across operators and vendors. More specifically, a 3GPP-registered AF uses an AF identifier per [1]; a third-party application not registered via the NEF may use a proprietary identifier scheme. This field shall not be NULL or empty. The value shall be consistent with the AF identity established during the authorization session. See note 2.
afIDFormat	string[1..1]	M	The format or namespace of the afIDContent field (e.g. "fqdn" per [1], "operator-assigned"). This forms the composite afID field together with the afIDContent field. This field shall not be NULL or empty. Allowed values include: "fqdn", "operator-assigned". Additional values may be defined by operator configuration. See note 2.
afSchemaVersion	string[1..1] (see [6])	M	Version of the serviceConfig schema. This shall conform to SemVer 2.0.0 (major.minor.patch[pre-release][+build] see [6]). This enables schema evolution across deployments.
messageID	string[1..1]	M	A per-request correlation identifier. The Network AI Agent shall echo this value unchanged in the corresponding response message. This field shall not be used as an idempotency key; the composite serviceID (serviceIDContent + serviceIDFormat) serves as the unique service identity. The scope of uniqueness of messageID is the AF session.
userIntentContent	string[1..1]	M	Value of the intent expression. This forms a composite key together with the userIntentFormat field. A composite key is necessary because there is no standard format applicable for all types of user intents. This field shall not be NULL or empty.
userIntentFormat	string[1..1]	M	The format or namespace of the userIntentContent field (e.g. natural-language-en, gsma-rcs-intent-1.0). This forms a composite key together with the userIntentContent field. This field shall not be NULL or empty.
userIntentSchemaVersion	string[1..1]	M	Version of the intent schema in use, expressed as a SemVer 2.0.0 string (see [6]). Enables receivers to route the intent to the correct parser without breaking older Network AI Agents. This field shall not be NULL or empty.
modalitySet	ModalityInfo[0..*] (see Table 5.2.2-3)	O	The set of content modalities accepted by the AF for service results. Each entry specifies a modality type and optional format constraint. When absent, the Network AI Agent selects the output modality based on the userIntent.
NOTE 1: A serviceID is composed of the serviceIDContent and serviceIDFormat fields, is locally unique within an AF scope. NOTE 2: The combination of an afID and a serviceID is globally unique.			

When an AF queries the information of a service, the Network AI Agent shall return the fields defined in Table 5.2.2-5 (AFServiceResult).

When a Network AI Agent invokes tools exposed by the AF, the AF acts as a tool server and the methods and data objects defined in clause 5.6.2 shall apply, with the AF in the tool server role and the Network AI Agent in the tool client role.

5.3.3 Protocol

Multi-agent communication spans multiple layers of the computer networking stack, with the critical intersection between autonomous AI agents and third-party applications localized at the application layer. Within this layer, architectural focus centres on two main vectors: data serialization efficiency and the design of extensible method-object frameworks that allow external entities to augment a Network AI Agent's capabilities. The following list contains selected options to implement the Network AI Agent-to-application interface that support the methods and data objects specified in clause 5.3.2.

NOTE 1: The list does not contain all possible implementation options. A detailed analysis of alternative transport protocols is left For Further Study (FFS).

Option 1: HTTP/1.1+JSON-REST

As shown in Figure 5.3.3-1, this is an HTTP/1.1 request sent from a Network AI Agent to a third-party application. This request consists of two primary parts, namely, the HTTP header, and the HTTP body. As HTTP/1.1 is a text-based protocol, the processor scans the incoming data byte-by-byte. First, it identifies the HTTP method as POST in the start-line.

The interface utilizes a custom method design pattern to execute lifecycle operations on resource collections. Rather than relying exclusively on standard Create, Read, Update, and Delete (CRUD) mapping via native HTTP verbs, specialized operations are explicitly designated within the URI path. This is reflected in the path (see "/api/v1/afService:create") that follows the custom method syntax (AIP-0136), where the resource path (/api/v1/afService) is separated from the action name ("create") by a colon (i.e. ':'). Route matching and parameter binding are performed at the application layer by the receiving application's HTTP server framework. This syntax shifts the functional routing command out of the HTTP body and directly into the HTTP request line, optimizing reverse-proxy routing and API gateway inspection.

Following this identification, the processor searches for the function associated with the route "/api/v1/afService:create" and performs parameter binding, extracting the JSON data from the HTTP body and storing it in memory for processing. All mandatory and optional fields defined in the AFServiceConfig (see Table 5.3.2-2) schema are serialized as flat parameters directly at the root level of the JSON request body.

```
POST /api/v1/afService:create HTTP/1.1
Host: agent.example.com
Content-Type: application/json
Accept: application/json
Content-Length: 529

{
    "serviceIDContent": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "serviceIDFormat": "UUID-v4",
    "afIDContent": "video-analytics-af.slice01.operator.com",
    "afIDFormat": "fqdn",
    "afSchemaVersion": "1.0.2",
    "messageID": ""c83a1523-2893-4a1d-9051-5b76b25121c8"",
    "userIntentContent": "Optimize downlink throughput for localized edge streaming",
    "userIntentFormat": "natural-language-en",
    "userIntentSchemaVersion": "2.0.0",
    "modalitySet": [
        {
            "modalityType": "video",
            "modalityFormat": "MP4"
        }
    ]
}
```

Figure 5.3.3-1: Structure of Network AI Agent-to-application request to create an AF service, when HTTP/1.1 with JSON-RPC 2.0 is used

Option 2: HTTP/1.1+JSON-RPC

Figure 5.3.3-2 shows the operation of a service creation as a JSON-RPC 2.0 request. In contrast to option 1, the method information is embedded into the HTTP body instead of HTTP header. Within the request message body, the AFServiceConfig fields are mapped entirely into the JSON-RPC "params" object, as required by the JSON-RPC 2.0 specification. The "Content-Length" header in the HTTP layer explicitly reflects the octet count of this stringified JSON payload. The composite serviceID is split into serviceIDContent, which specifies the unique identifier value, and serviceIDFormat, which explicitly identifies the encoding schema as UUID-v4. Similarly, the composite application function identifier is decomposed into afIDContent, containing the functional string, and afIDFormat, identifying it as a fully qualified domain name, ensuring that the receiving entity can correctly parse identities originating from either 3GPP-compliant or third-party proprietary systems.

```
POST /api/v1/rpc HTTP/1.1
Host: agent.example.com
Content-Type: application/json
Accept: application/json
Content-Length: 642

{
    "jsonrpc": "2.0",
    "method": "afService/create",
    "id": 101,
    "params": {
        "serviceIDContent": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
        "serviceIDFormat": "UUID-v4",
        "afIDContent": "video-analytics-af.slice01.operator.com",
        "afIDFormat": "fqdn",
        "afSchemaVersion": "1.0.2",
        "messageID": ""c83a1523-2893-4a1d-9051-5b76b25121c8"",
        "userIntentContent": "Optimize downlink throughput for localized edge streaming",
        "userIntentFormat": "natural-language-en",
        "userIntentSchemaVersion": "2.0.0",
        "modalitySet": [
            {
                "modalityType": "video",
                "modalityFormat": "MP4"
            }
        ]
    }
}
```

Figure 5.3.3-2: Structure of a Network AI Agent-to-application request to create an AF service, when HTTP/1.1 with JSON-RPC 2.0 is used

Option 3: HTTP/2 + JSON-RPC

Figure 5.3.3-3 illustrates the same JSON-RPC request to create an AF service as in Option 2. Different than the example in Figure 5.3.3-2, it uses HTTP/2. The HTTP/2 header employs pseudo headers with HPACK header compression, which is a more bandwidth-efficient alternative of encoding HTTP header in plain-text, as it is done when HTTP/1 is used IETF RFC 7541 [i.6].

NOTE 2: As defined in IETF RFC 7541 [i.6], HPACK adopts a hard-coded list of 61 frequently used headers and values that both the client and the server. Because of this static table, the protocol frequently replaces entire pseudo-headers with a single byte index number.

```
:method: POST
:path: /api/v1/rpc
:scheme: https
:authority: agent.example.com
content-type: application/json
accept: application/json
content-length: 642
{
    "jsonrpc": "2.0",
    "method": "afService/create",
    "id": 101,
    "params": {
        "serviceIDContent": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
        "serviceIDFormat": "UUID-v4",
        "afIDContent": "video-analytics-af.slice01.operator.com",
        "afIDFormat": "fqdn",
        "afSchemaVersion": "1.0.2",
        "messageID": ""c83a1523-2893-4a1d-9051-5b76b25121c8"",
        "userIntentContent": "Optimize downlink throughput for localized edge streaming",
        "userIntentFormat": "natural-language-en",
        "userIntentSchemaVersion": "2.0.0",
        "modalitySet": [
            {
                "modalityType": "video",
                "modalityFormat": "MP4"
            }
        ]
    }
}
```

Figure 5.3.3-3: Structure of a Network AI Agent-to-application request to create an AF service, when HTTP/2 with JSON-RPC 2.0 is used

The HTTP body is identical to the example in Figure 5.3.3-2, which is compliant with the JSON-RPC 2.0 specification, evident by the same content length of 642 octets. The mandatory and optional fields of the AFServiceConfig are embedded into the "params" object.

Option 4: HTTP/3 + JSON-RPC

This option assumes JSON-RPC serialization in the HTTP body to create an AF service as in the previous options, but utilizes HTTP/3. While it maintains the identical lowercase pseudo-header semantics and embedded JSON-RPC body payload as seen in Figure 5.3.3-3, the underlying transport framework is transitioned from TCP to the QUIC protocol. By operating natively over UDP-based QUIC streams, this configuration entirely eliminates transport-layer head-of-line blocking. Consequently, packet drop or network degradation on adjacent streams does not stall the transmission or processing of the service creation request. To accommodate QUIC's out-of-order packet delivery profile, HTTP/3 replaces HPACK with QPACK header compression to avoid decompression blocking [i.20].

NOTE 3: In contrast to HPACK, QPACK utilizes an expanded static table containing 99 pre-defined header fields and values instead of 61. It offloads dynamic table synchronization to dedicated, unidirectional control streams, maintaining maximum bandwidth efficiency and stream independence between the Network AI Agent and the application function.

Option 5: HTTP/2 + gRPC

gRPC constitutes another option to implement the Network AI Agent-to-application interface specified in clause 5.3.2. Thanks to its binary encoding mechanism based on protocol buffers (i.e. .proto), gRPC replaces the plain-text strings present in the JSON-RPC request body, such as the key names and curly brackets. This allows the client Network AI Agent to reduce the payload through efficient encoding mechanism.

The method names shall be encoded as the "method" parameter of the JSON-RPC 2.0 request object (see [3]).

5.3.4 Security

5.3.4.1 Authentication

Authentication of Application Functions accessing the interface defined in clause 5.3 of the present document shall comply with clause 5.8.2.3 of the present document. An AF shall authenticate using mutual TLS (mTLS) with a valid client certificate issued or validated by an operator-trusted certificate authority. A Network AI Agent shall reject any connection from an AF that fails mTLS authentication.

For AF-hosted tool invocations, the AF shall similarly authenticate the Network AI Agent via mTLS and reject any connection that fails validation.

5.3.4.2 Authorization

Authorization of Application Functions shall comply with clause 5.8.3.2 of the present document. An AF shall obtain an OAuth 2.0 access token from the operator's designated authorization server (e.g. CAPIF Core Function or NRF) prior to invoking any method via the Network Exposure Function (NEF) as defined in clause 5.3.2 of the present document. The access token shall be scoped to the specific method and Network AI Agent being accessed.

The following authorization requirements apply per method:

1) afService/create: requires a valid access token scoped to service creation for the requesting AF. The Network AI Agent shall verify that the afID carried in AFServiceConfig matches the identity asserted in the access token.

2) afService/update: requires a valid access token scoped to service update for the requesting AF, and shall only succeed if the service identified by the composite serviceID was previously created by the same AF.

3) afService/get: requires a valid access token scoped to service retrieval. The Network AI Agent shall only return information pertaining to services created by the requesting AF.

4) afService/delete: requires a valid access token scoped to service deletion, and shall only succeed if the service was created by the requesting AF.

5) Tool invocation (clause 5.6.2 of the present document as applied to AF-hosted tools): requires a valid access token scoped to tool invocation for the specific tool URI. An AF acting as a tool server shall enforce access control on tool exposure, permitting invocation only by authenticated and authorized Network AI Agents.

5.3.4.3 Data Access Constraints

An AF shall not be granted access to Class 3 or Class 4 data as defined in clause 5.8.3.5 of the present document via the interface defined in clause 5.3 of the present document. Service context information returned in response to afService/get shall be filtered to exclude any subscriber-specific data not pertaining to the requesting AF's own service context.

5.3.4.4 Transport Security

Transport security for the AF interface shall comply with clause 5.8.4 of the present document. All AF interface interactions utilizing HTTP/3 shall rely on the native transport security provided by the QUIC protocol, which integrates TLS 1.3 cryptographic handshakes. Fallback transport options shall be carried over TLS 1.3 or later.

5.4 Interface between Network AI Agent and Agent Repository

5.4.1 Requirements

The network shall support the deployment of multiple Network AI Agents, each with distinct roles and capabilities. To enable collaboration between multiple Network AI Agents, the network shall provide a mechanism by which Network AI Agents may discover each other. This method shall enable a Network AI Agent to identify one or more suitable peer Network AI Agents for collaborative task execution. To support this, a network function called the Agent Repository Function (ARF) is used. The ARF is defined in clause 3.1 of the present document.

The ARF is responsible for storing and maintaining lifecycle information about registered Network AI Agents. This information may be static (e.g. agent identity and type) or dynamic (e.g. current availability, capabilities, and endpoint address). Therefore, the ARF shall allow Network AI Agents to:

- register Network AI Agent information that is stored by ARF;
- discover other Network AI Agents by querying the ARF for registered Network AI Agent profiles that match specific criteria;
- update registered Network AI Agent information when that information becomes outdated or changes; and
- deregister a Network AI Agent from the ARF when that Network AI Agent is no longer active.

The entity that provides information to the ARF or requests information from the ARF may differ from the Network AI Agent being registered. For example, an intermediate management entity may register a Network AI Agent on its behalf. In this case, the intermediate entity does not have to be a Network AI Agent.

The ARF interface is independent of the internal realization of a registered Network AI Agent. For instance, a single registered Network AI Agent entry in the ARF may represent a group of Network AI Agents that act as a single logical entity. In addition, that single logical entity may have its own internal mechanisms for workflow orchestration and task execution.

5.4.2 Methods and Data Objects

The ARF shall expose an interface to Network AI Agents that supports the following four methods defined in Table 5.4.2-1.

Table 5.4.2-1: Supported methods for the interface between a Network AI Agent and the ARF

Method Name	Description	Inputs	Outputs
agent/register	Register a new Network AI Agent with the ARF. This shall fail if a Network AI Agent with the specified composite AgentID (i.e. agentIDContent + agentIDFormat) is already registered.	AgentProfile as defined in Table 5.4.2-2.	RegistrationResult object as defined in Table 5.4.2-14.
agent/discover	Enable a Network AI Agent to discover a set of Network AI Agents with specific skills or capabilities or features.	AgentDiscoveryFilter as defined in Table 5.4.2-3.	AgentDiscoveryResult as defined in Table 5.4.2-3a.
agent/update	Update the mutable fields of an existing Network AI Agent profile. This shall fail if no Network AI Agent with the specified agentID is registered. The agentID and plmnID fields shall both be immutable after registration.	agentIDContent : string[1..1] and agentIDFormat : string[1..1], identifying the Network AI Agent to be updated, plus a partial AgentProfile containing only the fields to be updated. The agentIDContent, agentIDFormat, and plmnID fields shall not be included in the partial AgentProfile, as these fields are immutable after registration.	OperationResult object as defined in Table 5.2.2-4.
agent/deregister	Deregister an existing Network AI Agent from the ARF. This shall fail if no Network AI Agent with the specified composite agentID is registered.	agentIDContent : string[1..1] and agentIDFormat : string[1..1], identifying the Network AI Agent to be deregistered.	OperationResult object as defined in Table 5.2.2-4.

The name field of the Network AI Agent shall not be used as a deregistration key. This is because the name field is not required to be unique within the ARF.

NOTE: The agent/deregister method accepts only AgentID (i.e. agentIDContent + agentIDFormat) as the deregistration key.

Network AI Agents shall refresh their registration details, as defined by the creationTime and expirationTime fields in Table 5.4.2-2, upon registration and prior to expiration using agent/update, respectively.

The agent information is recorded in the AgentProfile, which includes the information in Table 5.4.2-2.

Table 5.4.2-2: Fields of the AgentProfile data type

Field Name	Type	Presence	Description
agentIDContent	string[1..1]	M	Value of the agentID identifier. This forms the composite agentID field together with the agentIDFormat field. This field shall not be NULL or empty. This field shall be immutable after the Network AI Agent is registered.
agentIDFormat	string[1..1]	M	The format or namespace of the agentIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite agentID field together with the agentIDContent field. This field shall not be NULL or empty. This field shall be immutable after the Network AI Agent is registered.
schemaVersion	string[1..1]	M	Version of the AgentProfile schema. This shall conform to SemVer 2.0.0 (major.minor.patch[pre-release][+build]). This enables schema evolution across deployments. See [6] for more information.
groupIDSet	string [0..*]	O	A set of identifiers of the persistent Network AI Agent groups that this Network AI Agent belongs to, where the group members are fixed.
funcType	AgentType[1..1]	M	The functional type of the Network AI Agent, used as a primary discovery discriminator. Allowed values are: "ai-inference", "connectivity", "data-processing", "multi-function", and "sensing". See note 1.
name	string[1..1]	M	A human-readable name of the Network AI Agent. This is not required to be unique within the ARF. Hence, this field shall not be used as a key in any ARF operation. When an AgentDiscoveryFilter specifies a name value, the ARF shall return all AgentProfile objects whose name field matches the filter value. The set of returned AgentProfile objects may be zero or more entries.
provider	string[0..1]	O	The name of the Network AI Agent provider. This is not required to be unique within the ARF. Hence, this field shall not be used as any type of key.
url	string[1..1]	M	The endpoint address at which the Network AI Agent is hosted. The URL format per [7] shall be used.
plmnID	string[1..1]	M	The public land mobile network identifier that the Network AI Agent belongs to. This field is immutable after registration. The ARF shall reject any agent/update request in which the partial AgentProfile includes a plmnID field. In addition, the ARF shall return an OperationResult with a status of "failure" and an appropriate error code. The format shall be defined by [9].
locationInfo	NetworkInfo[0..1]	O	A custom data type that defines the geographical location information of the Network AI Agent. See Table 5.4.2-12.
description	string[1..1]	M	A human-readable overall description of the Network AI Agent.
skillSetInfo	SkillInfo[1..*]	M	A custom data type that defines the skill name, version, and description of each skill. See Table 5.4.2-6. In addition, see note 2.
capabilitySetInfo	CapabilityInfo[0..*]	O	A custom data type that defines the protocol capabilities supported by the Network AI Agent. See Table 5.4.2-7.
securitySetInfo	SecurityInfo[1..*]	M	A custom data type that defines the security scheme(s) used to authenticate the Network AI Agent. See Table 5.4.2-8. In addition, see note 3.
methodSet	MethodInfo[1..*]	M	The interface method(s) provided by the Network AI Agent. See Table 5.4.2-9.
inputSchemaSet	InputSchemaInfo[0..*]	O	The input schema(s) to the Network AI Agent. See Table 5.4.2-10.
outputSchemaSet	OutputSchemaInfo[0..*]	O	The output schema(s) generated by the Network AI Agent. See Table 5.4.2-11.

Field Name	Type	Presence	Description
creationTime	string[1..1]	M	ISO 8601 [14] UTC datetime, e.g. "2026-05-4T21:39:00Z". This shall include explicit UTC offset (Z or +00:00). This is set by the ARF at registration time. This field shall not be supplied or modified by the registering Network AI Agent.
expirationTime	string[1..1]	M	ISO 8601 [14] UTC datetime. Shall be later than creationTime. This is supplied by the registering Network AI Agent. This may be updated via agent/update prior to expiry. Expiry handling is specified in the normative paragraphs following this table.
NOTE 1: The "multi-function" value is used when the Network AI Agent supports capabilities belonging to more than one functional type. In such cases, the Skill information field enumerates the specific capability types supported. NOTE 2: Table 5.4.2-3 defines the AgentDiscoveryFilter object used as input to the agent/discover method defined in Table 5.4.2-1. NOTE 3: Table 5.4.2-6 defines the fields of the SkillInfo data type.			

An agent registration whose expirationTime field value is in the past shall be treated by the ARF as expired. The ARF shall not return an expired AgentProfile in response to an agent/discover request.

An expired registration shall not be automatically deregistered by the ARF; the registering entity shall explicitly invoke agent/deregister to remove the registration, or agent/update to refresh the expirationTime field to a future value.

NOTE 1: Security scheme objects in AgentProfile are used by discovering Network AI Agents to establish authenticated connections. Discovery responses do not include credential values (e.g. private keys or bearer tokens); only scheme metadata is included.

NOTE 2: Handling of abandoned registrations (i.e. expired registrations that are neither refreshed nor explicitly deregistered) is for further study. Candidate behaviours include automatic deregistration after a configurable grace period and permanent retention with expired status.

Table 5.4.2-3 defines the AgentDiscoveryFilter object. It is used as an input to the agent/discover method defined in Table 5.4.2-1.

Table 5.4.2-3: Fields of the AgentDiscoveryFilter object

Field Name	Type	Presence	Description
allowedType	AgentType[0..*]	O	Filter by the functional type of the Network AI Agent, as defined by the funcType field in Table 5.4.2-2.
skillSetInfo	SkillFilter[0..*]	O	Filter by one or more skill keywords (see Table 5.4.2-4).
plmnID	string[0..1]	O	Filter by PLMN ID as defined in Table 5.4.2-2. If this field is absent, then Network AI Agents from all PLMNs accessible to the requesting Network AI Agent may be returned.
capabilitySet	CapabilityFilter[0..*]	O	Filter by one or more required protocol capability strings.
maxResultSet	Integer[0..1]	O	This defines the maximum number of AgentProfile objects to return. If absent, the ARF may apply an operator-configured default limit.

At least one field shall be present in an AgentDiscoveryFilter. The ARF shall reject an agent/discover request in which all AgentDiscoveryFilter fields are absent. In addition, the ARF shall return an OperationResult with status "failure" and an error code indicating that no filter criteria were specified.

Table 5.4.2-3a defines the AgentDiscoveryResult data type.

Table 5.4.2-3a: Fields of the AgentDiscoveryResult data type

Field Name	Type	Presence	Description
totalCount	integer[1..1]	M	Total number of AgentProfile objects returned in this response.
agentProfileSet	AgentProfile[0..*]	O	The set of AgentProfile objects matching the AgentDiscoveryFilter. This shall be empty when no registered agents match the filter criteria. The ARF shall not include AgentProfile objects for which the requesting entity lacks discovery authorization, as specified in clause 5.4.4.3.

Table 5.4.2-4 defines the SkillFilter data type.

Table 5.4.2-4: Fields of the SkillFilter data type

Field Name	Type	Presence	Description
skillName	string[1..1]	M	Match Network AI Agent advertising this skill name. This is an exact match.
minSkillVersion	string[0..1]	O	If present, match only Network AI Agents advertising a skillVersion greater than or equal to this SemVer 2.0.0 string [6].

Table 5.4.2-5 defines the CapabilityFilter data type.

Table 5.4.2-5: Fields of the CapabilityFilter data type

Field Name	Type	Presence	Description
capabilityName	string[1..1]	M	Match Network AI Agents advertising this capability name. This is an exact match against the capabilityName field in CapabilityInfo (Table 5.4.2-7).
minCapabilityVersion	string[0..1]	O	If present, match only Network AI Agents advertising a capabilityVersion greater than or equal to this SemVer 2.0.0 string [6].

Table 5.4.2-6 defines the fields of the SkillInfo data type.

Table 5.4.2-6: Fields of the SkillInfo data type

Field Name	Type	Presence	Description
skillName	string[1..1]	M	Machine-readable skill identifier. Shall use lowercase kebab-case (e.g. "qos-optimization", "mobility-prediction"). Shall not be null or empty.
skillVersion	string[1..1]	M	Version of the skill implementation, expressed as a SemVer 2.0.0 string [6]. Enables version-aware discovery filtering.
skillDescription	string[0..1]	O	Human-readable description of the skill. Informative only; shall not be used for programmatic filtering.

Table 5.4.2-7 defines the fields of the CapabilityInfo data type.

Table 5.4.2-7: Fields of the CapabilityInfo data type

Field Name	Type	Presence	Description
capabilityName	string[1..1]	M	Machine-readable protocol capability identifier (e.g. http2-grpc, json-rpc-2.0). Shall use lowercase kebab-case. Shall not be null or empty.
capabilityVersion	string[0..1]	O	Version of the capability, expressed as a SemVer 2.0.0 string [6]. Enables version-aware discovery filtering.

Table 5.4.2-8 defines the fields of the SecurityInfo data type.

Table 5.4.2-8: Fields of the SecurityInfo data type

Field Name	Type	Presence	Description
securityName	string[1..1]	M	Machine-readable security scheme identifier. Shall use lowercase kebab-case (e.g. mtls, oauth2-client-credentials, oauth2-bearer). Shall not be null or empty.
securityVersion	string[1..1]	M	Version of the security scheme implementation, expressed as a SemVer 2.0.0 string. Enables version-aware discovery filtering.
securityDescription	string[0..1]	O	Human-readable description of the security scheme. Informative only; shall not be used for programmatic filtering.

Table 5.4.2-9 defines the fields of the MethodInfo data type.

Table 5.4.2-9: Fields of the MethodInfo data type

Field Name	Type	Presence	Description
methodName	string[1..1]	M	Machine-readable method name. Shall not be null or empty.
methodVersion	string[1..1]	M	Version of the method implementation, expressed as a SemVer 2.0.0 string. Enables version-aware discovery filtering.
methodDescription	string[0..1]	O	Human-readable description of the method. Informative only; shall not be used for programmatic filtering.

Table 5.4.2-10 defines the fields of the InputSchemaInfo data type.

Table 5.4.2-10: Fields of the InputSchemaInfo data type

Field Name	Type	Presence	Description
inputSchemaURI	string[1..1]	M	A URI reference to an external schema document (e.g. JSON Schema or a Protobuf descriptor).
inputSchemaFormat	string[1..1]	M	Identifies the schema language. Allowed values: json-schema-draft-07, json-schema-draft-2020-12, proto3, openapi-3.1. Additional values may be defined by operator configuration.

Table 5.4.2-11 defines the fields of the OutputSchemaInfo data type.

Table 5.4.2-11: Fields of the OutputSchemaInfo data type

Field Name	Type	Presence	Description
outputSchemaURI	string[1..1]	M	A URI reference to an external schema document (e.g. JSON Schema or a Protobuf descriptor).
outputSchemaFormat	string[1..1]	M	Identifies the schema language. Allowed values: json-schema-draft-07, json-schema-draft-2020-12, proto3, openapi-3.1. Additional values may be defined by operator configuration.

Table 5.4.2-12 defines the fields of the NetworkInfo data type.

Table 5.4.2-12: Fields of the NetworkInfo data type

Field Name	Type	Presence	Description
plmnID	string[0..1]	O	The public land mobile network identifier that the Network AI Agent belongs to. The format shall be defined by [9].
dataCentreID	DataCentreInfo[0..1]	O	A custom data type that identifies a data centre. See Table 5.4.2-13.
NOTE: The plmnID inside NetworkLocation is distinct from plmnID in the top-level AgentProfile. The top-level one identifies which PLMN the Network AI Agent belongs to (administrative domain); the NetworkLocation.plmnID identifies which PLMN's physical infrastructure the Network AI Agent is hosted in. The values of these fields typically differ in roaming or multi-operator deployment scenarios.			

Table 5.4.2-13 defines the fields of the DataCentreInfo data type.

Table 5.4.2-13: Fields of the DataCentreInfo data type

Field Name	Type	Presence	Description
dcContent	string[1..1]	M	Value of the data centre identifier. This forms the composite dcID field together with the dcFormat field. This field shall not be NULL or empty. This field shall be immutable after the Network AI Agent is registered.
dcFormat	string[1..1]	M	The format or namespace of the dcContent field (e.g. nfvi-pop-id, cloud-zone, operator-assigned). This forms the composite dcID field together with the dcContent field. This field shall not be NULL or empty. This field shall be immutable after the Network AI Agent is registered.

Table 5.4.2-14 defines the fields of the RegistrationResult data type.

Table 5.4.2-14: Fields of the RegistrationResult data type

Field Name	Type	Presence	Description
status	enum[1..1]	M	The status can either be "success" or "failure".
creationTime	string[0..1]	O	ISO 8601 [14] UTC datetime assigned by the ARF at registration. Present only when status is "success". The registering Network AI Agent shall use this value as the authoritative creation timestamp.
errorCode	string[0..1]	O	Machine-readable error code. Present only when status is "failure".
errorMessage	string[0..1]	O	Human-readable error description. Present only when status is "failure".

5.4.3 Protocol

The ARF is an operator-internal network function. Communication between a Network AI Agent and the ARF shall use either the HTTP/2+gRPC or HTTP/3+gRPC protocol bindings specified in clause 5.3.3, with HTTP/3+gRPC as the preferred binding consistent with the general protocol recommendation of the present document.

NOTE: The ARF interface is internal to the operator's 5G or 6G core network and is not exposed externally. The HTTP/3+gRPC binding recommendation is subject to the same QUIC firewall traversal caveat noted in clause 5.3.3. If QUIC is not supported by the operator's internal network path, HTTP/2+gRPC is used instead.

The JSON-RPC 2.0 message encoding defined in clause 5.3.3 (Option 2) applies to the method invocations defined in Table 5.4.2-1. The method names shall be encoded as the "method" parameter of the JSON-RPC 2.0 request object (see [3]).

5.4.4 Security

5.4.4.1 Authentication

Authentication of Network AI Agents and management entities accessing the interface defined in clause 5.4 of the present document shall comply with clause 5.8.2.4 of the present document. Each Network AI Agent or operator-authorized management entity shall authenticate to the ARF using mutual TLS (mTLS) with a valid client certificate issued or validated by an operator-trusted certificate authority, consistent with the Service-Based Interface (SBI) security architecture defined in 3GPP TS 33.501 [5].

The ARF shall reject any connection from a client that fails mTLS authentication. The ARF shall present a valid TLS server certificate; the connecting Network AI Agent shall verify this certificate against the operator-trusted certificate authority before transmitting any registration, update, discovery, or deregistration request.

For Network AI Agents operating within the 5G Service Based Architecture, an OAuth 2.0 access token shall additionally be obtained from the Network Repository Function (NRF) using the client credentials grant, in accordance with 3GPP TS 33.501 [5]. This shall be presented to the ARF with each request. The ARF shall validate the NRF-issued token before processing any request.

NOTE: The entity interacting with the ARF acts either as the Network AI Agent being registered or as a distinct management entity. An operator-authorized management entity performs registration, update, or deregistration of a Network AI Agent on its behalf, once that entity has successfully completed mTLS authentication and holds the appropriate operator authorization.

5.4.4.2 Authorization

1) Agent/register: requires that the requesting entity holds a valid registration authorization for the ARF, scoped to agent registration within the operator's PLMN. The ARF shall verify that the agentIDContent and agentIDFormat in the AgentProfile are consistent with the authenticated identity of the requesting entity.

2) Agent/discover: requires that the requesting entity holds a valid read authorization for the ARF. The ARF shall filter the returned AgentProfile set to include only agents for which the requesting entity holds discovery authorization, as specified in clause 5.4.4.3.

3) Agent/update: requires that the requesting entity holds update authorization for the specific AgentProfile identified by the composite agentID. An entity shall only be authorized to update the AgentProfile of an agent that it registered or that it is explicitly authorized to manage by operator policy.

4) Agent/deregister: requires that the requesting entity holds deregistration authorization for the specific AgentProfile identified by the composite agentID. An entity shall only be authorized to deregister an agent that it registered or that it is explicitly authorized to manage by operator policy.

Authorization of Network AI Agents and management entities shall comply with clauses 5.8.3.3 and 5.8.5 of the present document. The following authorization requirements apply per method:

1) Agent/register: This requires that the requesting entity holds a valid registration authorization for the ARF, scoped to agent registration within the operator's PLMN. The ARF shall verify that the composite agentID (i.e. the agentIDContent and agentIDFormat fields) carried in the AgentProfile are consistent with the authenticated identity of the requesting entity. The ARF shall reject any registration request in which the authenticated client identity is not consistent with the agentIDContent and agentIDFormat values in the AgentProfile, unless the client is an operator-authorized management entity.

2) Agent/update: This requires that the requesting entity holds update authorization for the specific AgentProfile identified by the composite agentID. An entity shall only be authorized to update the AgentProfile of an agent that it registered or that it is explicitly authorized to manage by operator policy. The ARF shall reject any attempt to modify the agentIDContent, agentIDFormat, or plmnID fields, as these are immutable after registration.

3) Agent/deregister: This requires that the requesting entity holds deregistration authorization for the specific AgentProfile identified by the composite agentID. An entity shall only be authorized to deregister an agent that it registered or that it is explicitly authorized to manage by operator policy.

4) Agent/discover: The requesting entity shall hold a be accessible to any Network AI Agent that has successfully completed mTLS authentication. valid read authorization for the ARF. The ARF shall filter the returned AgentProfile set to include only agents for which the requesting entity holds discovery authorization, as specified in clause 5.4.4.3. The ARF shall not require additional OAuth 2.0 scoping for discovery requests beyond successful mTLS authentication, unless operator policy mandates additional access control on discovery.

5.4.4.3 Data Access Constraints

The ARF shall filter the AgentProfile objects returned in response to an agent/discover request to include only Network AI Agents that the requesting entity is authorized to discover, based on the requester's authenticated identity and operator-configured discovery authorization policy as defined in clause 5.8.5. The ARF shall not disclose the existence of Network AI Agents for which the requesting entity holds no discovery authorization.

The following AgentProfile field classifications apply for the purposes of data access constraints:

1) Class 1 (public within operator domain): agentIDContent, agentIDFormat, name, funcType, schemaVersion.

2) Class 2 (restricted, requires read authorization): skillSet, supportedMethods, groupIDSet, endpointSet.

3) Class 3 (sensitive, requires explicit operator authorization): plmnID, creationTime, expirationTime.

4) Class 4 (operator-confidential, not releasable via discovery): securitySchemes.

The ARF shall not return Class 3 fields in a discovery response unless the requesting entity holds explicit operator authorization for those fields. Class 3 fields shall be returned in full only in response to agent/register confirmation (to the registering entity) and agent/update confirmation (to the updating entity).

The ARF shall not return Class 4 fields in any agent/discover response, regardless of the requesting entity's authorization level. Class 4 fields shall be accessible only to the registering entity via the agent/register confirmation response and to operator-authorized management entities via agent/update. Security scheme credential values (e.g. private keys, bearer tokens) shall not be included in discovery responses; only security scheme metadata as defined in Table 5.4.2-8 shall be returned.

5.4.4.4 Transport Security

Transport security for the ARF interface shall comply with clause 5.8.4 of the present document. All interactions on the interface defined in clause 5.4 shall be carried over TLS 1.3 or later. TLS 1.2 fallback as permitted by clause 5.8.4 should not be used for the ARF interface given the sensitivity of the AgentProfile data carried, including agent endpoints, security schemes, and PLMN identifiers; operator deployments shall mandate TLS 1.3 for this interface.

The preferred protocol binding is HTTP/3+gRPC as specified in clause 5.4.3, which provides native TLS 1.3 via QUIC. Where HTTP/2+gRPC is used as the fallback binding, TLS 1.3 shall be used as the underlying transport security mechanism.

NOTE: The prohibition on HTTP/1.1 for ARF methods is more restrictive than the general protocol preference stated in clause 5.4.3. This restriction applies specifically because AgentProfile registration and update payloads contain security scheme metadata whose exposure over unencrypted or weakly protected transports would undermine the integrity of the agent authentication framework.

5.5 Interface among Network AI Agents

5.5.1 Requirements

The next generation mobile network shall support seamless communication among Network AI Agents and facilitate multi-agent collaboration towards fulfilling user intents. In particular, the mobile network shall provide methods that allow Network AI Agents to autonomously create, orchestrate, and execute tasks. The network shall enable agent discovery to facilitate identification of Network AI Agents with required skills and capabilities, as specified in clause 5.4. The mobile network shall introduce mechanisms to collect, store, and track the history of interactions, partial results, and the status of executed service to ensure traceability and auditability.

Thus, the interface between Network AI Agents shall support:

- creation of a new task, e.g. upon a new service request from UE or application;
- update of an ongoing task, for instance, due to a change in the corresponding service request, including task cancellations;
- information retrieval (e.g. on-demand or subscription-based) about the status of an ongoing task;
- subscription management to add or remove subscriptions for ongoing tasks.

5.5.2 Methods and Data Objects

According to the requirements described in clause 5.5.1, the interface among Network AI Agents shall support the following methods.

Table 5.5.2-1: Supported methods by the interface between Network AI Agents

Method Name	Description	Inputs	Outputs
task/create	Create a new task. Shall fail if a task with the specified composite taskID already exists.	Fields as shown in Table 5.5.2-2.	On success: • TaskCreateResult as defined in Table 5.5.2-4a. On error: • TaskOperationError as defined in Table 5.5.2-8 (see note 2)
task/update	Update an existing task.	Fields as shown in Table 5.5.2-2.	On success: • TaskUpdateResult as defined in Table 5.5.2-4b. On failure: • TaskOperationError as defined inTable 5.5.2-8. (see note 2)

Method Name	Description	Inputs	Outputs
task/get	Query the current information, e.g. status or result of a task.	agentIDContent : string[1..1], agentIDFormat : string[1..1], taskIDContent : string[1..1], taskIDFormat : string[1..1], messageID : string[1..1]	On success: • Fields as shown in Table 5.5.2-3. On failure: • TaskOperationError as defined in Table 5.5.2-8. (see note 2)
task/delete	Delete an existing task, including all of its context.	agentIDContent : string[1..1], agentIDFormat : string[1..1], taskIDContent : string[1..1], taskIDFormat : string[1..1], messageID : string[1..1]	On success: • TaskDeleteResult as defined in Table 5.5.2-4c. On failure: • TaskOperationError as defined in Table 5.5.2-8. (see note 2)
task/subscribe	Subscribe to updates about the status of an ongoing task.	Fields as shown in Table 5.5.2-5a.	On Success: • TaskSubscribeResult as defined in Table 5.5.2-5b. On failure: • TaskOperationError as defined in Table 5.5.2-8. (see note 2)
task/unsubscribe	Unsubscribe to receiving updates about the status of an ongoing task.	Fields as shown in Table 5.5.2-6a.	On Success: • TaskUnsubscribeResult as defined in Table 5.5.2-6b. On failure: • TaskOperationError as defined in Table 5.5.2-8. (see note 2)
task/notify	Publish information about the status of an ongoing task to all of its subscribers.	Fields as shown in Table 5.5.2-3	NA

NOTE 1: A Network AI Agent receiving a request carrying any of the methods task/create, task/update, task/get, task/delete, task/subscribe, or task/unsubscribe responds synchronously with a corresponding response message. Asynchronous task status updates are delivered via task/notify.

NOTE 2: The distinction between a successful (i.e. "on success") case and a failed operation (i.e. "on failure") case depends on the protocol binding. For instance, if JSON-RPC is used, a JSON-RPC response message to a successful operation contains a "result" object, while a failed operation contains an "error" object (and no "result" object). The tables in clause 5.5.2 assume that this distinction is implementation-specific and handled by the protocol binding. Table 5.5.2-2 through Table 5.5.2-8 specify the fields given the success/failure of the operation.

When a Network AI Agent requests another Network AI Agent to create or update a task, the fields in Table 5.5.2-2 shall be included in the request message.

Table 5.5.2-2: Fields of inputs for the task/create and task/update methods

Field Name	Type	Presence	Description
agentIDContent	string[1..1]	M	Value of the AgentID identifier. This forms the composite AgentID field together with the AgentIDFormat field. This field shall not be NULL or empty. The composite Agent ID is used by the receiving agent to evaluate whether the requesting agent is authorized to create a new task or update the specified task. The authorization policy may also depend on the operator's configuration. However, that is out of scope for the present document.
agentIDFormat	string[1..1]	M	The format or namespace of the AgentIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite AgentID field together with the AgentIDContent field. This field shall not be NULL or empty.
serviceContextID	string[0..1]	O	Identifier of the service context associated with this service. This identifier can be used to correlate related service and task invocations across interfaces. The serviceContextID is present only when a service context has been created for the service.
serviceContext	ContextInfo[0..1]	O	Optional context associated with this service, such as UE session attributes, UE application state, or slice information. When present, the serviceContext shall be encoded as a ContextInfo object as defined in Table 5.2.2-5a.
taskIDContent	string[1..1]	M	Value of the taskID identifier. This forms the composite taskID field together with the taskIDFormat field. A composite taskID is a per-task correlation identifier created by the client/initiator Network AI Agent. The receiving agent shall treat the composite of the requesting agent's identifier and this composite taskID as the unique key for task lookup in its internal namespace. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	A per-request correlation identifier. The responding agent shall populate this field by echoing the message identifier value from the corresponding request message without modification. The scope of uniqueness of messageID is the Network AI Agent that creates or updates the task.
taskInput	Part[1..*]	M	Input data for the task, expressed as one or more Part objects. See Table 5.6.1.2-4 for the definition of the Part data type. This field shall not be NULL or empty.
NOTE 1: The service context includes all data and information relevant to the service, including the UE intent, the Network AI Agents associated with the service, and previous message exchanges between the UE and agents. Access to service context or part of it by the Network AI Agent requires prior authorization as specified in clause 5.8. NOTE 2: Business rules, SLAs and SLSs, and other relevant information is also typically part of context. See [4] and [8] for additional information.			

When a Network AI Agent requests to query the status of an existing task or subscribes to receive updates about a task of another agent, the fields of the response message are presented in Table 5.5.2-3.

Table 5.5.2-3: Information of outputs for the task/get and inputs for task/notify method

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the taskID identifier. This forms the composite taskID field together with the taskIDFormat field. A composite taskID is a per-task correlation identifier from the corresponding task/get or task/subscribe request that caused the transmission of the task/get response or task/notify, respectively. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	A per-request correlation identifier. The responding agent shall populate this field by echoing the message identifier value from the corresponding request message without modification.
serviceContextID	string[0..1]	O	Identifier of the service context associated with this service. This identifier can be used to correlate related service and task invocations across interfaces. The serviceContextID is present only when a service context has been created for the service.
serviceContext	ContextInfo[0..1]	O	Optional context associated with this service, such as UE session attributes, UE application state, or slice information. When present, the serviceContext shall be encoded as a ContextInfo object as defined in Table 5.2.2-5a.
timestamp	string[1..1]	M	Current system time at which this response was generated. Shall be expressed as an ISO 8601 [14] UTC datetime string (e.g. "2026-05-15T17:00:00Z"). Shall include an explicit UTC offset (Z or +00:00).
taskState	TaskState[1..1]	M	Current state of the task. See TaskState enumeration in Table 5.5.2-7.
taskStateMessage	string[0..1]	O	Descriptive information containing further details about the task state. This field is intended to contain explanatory information, e.g. when in "failed", "cancelled", "inputRequired", and "authorizationRequired" states.
taskResult	Part[0..*]	O	Partial or final results generated for the task, expressed as a set of Part objects. See Table 5.6.1.2-4 for the definition of the Part data type.
NOTE 1: The service context means all the information relevant to the service, including the UE intent, the Network AI Agents associated with the service, and previous message exchange between the UE and the Network AI Agents. NOTE 2: Business rules, SLAs and SLSs, and other relevant information is also typically part of context. See ETSI GS ENI 005 [4] and ETSI GS ENI 019 [8] for additional information.			

In case of the response to a task/get request, Table 5.5.2-3 specifies the fields of a successful operation, which is independent of the execution of a task. The requested task may be in a failed state, and this shall be conveyed to the requester via the message defined in Table 5.5.2-3. However, in cases such as when the task associated with the requested composite taskID is not found in the receiving agent's database, when the request syntax is wrong, or when the request contains invalid input parameters according to Table 5.5.2-3, then the response complies with the TaskOperationError defined in Table 5.5.2-8.

When a Network AI Agent receives a task/create request from another Network AI Agent, the response message after a successful operation contains the fields presented in Table 5.5.2-4a.

Table 5.5.2-4a: Fields of the TaskCreateResult

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the taskID identifier confirmed by the responding/serving agent. This shall be the same taskIDContent value from the corresponding task/create message. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The Network AI Agent shall echo this value unchanged in the corresponding response message. The scope of uniqueness of messageID is the Network AI Agent that creates the task.
taskState	TaskState[1..1] (see Table 5.5.2-7)	M	Immediate task state after creation attempt. This field shall not be NULL or empty.
taskStateMessage	string[0..1]	O	Descriptive information containing further details about the task state. This field is intended to contain explanatory information, e.g. when in "failed", "cancelled", "inputRequired", and "authorizationRequired" states.

A "task/create" request may fail due to invalid fields in the request, syntax/type errors, and internal errors of the receiving agent, e.g. resource scarcity. Each of these errors lead to failed creation of a new task with the requested composite taskID, thus, the task creation request has no impact on the receiving Network AI Agent side after it has been processed. These operation errors are captured by the TaskOperationError defined in Table 5.5.2-8.

After its creation or modification, A task can enter an interrupted state such as "inputRequired" and "authorizationRequired" as defined in Table 5.5.2-7 or a terminal state such as "failed" or "cancelled". These shall not trigger a TaskOperationError. These cases are handled by taskState and taskStateMessage fields, e.g. in Table 5.5.2-4a and Table 5.5.2-4b.

When a Network AI Agent receives a task/update request from another Network AI Agent, the response message after a successful operation contains the fields presented in Table 5.5.2-4b.

Table 5.5.2-4b: Fields of the TaskUpdateResult

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the taskID identifier confirmed by the responding/serving agent. This shall be the same taskIDContent value from the corresponding task/update message. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.
taskState	TaskState[1..1]	M	Immediate task state after task update attempt.
taskStateMessage	string[0..1]	O	Descriptive information containing further details about the task state. This field is intended to contain explanatory information, e.g. when in "failed", "cancelled", "inputRequired", and "authorizationRequired" states.

A "task/update" request may fail due to invalid fields in the request, syntax/type errors, and internal errors of the receiving agent, e.g. resource scarcity. Each of these errors lead to failed operation, triggering a TaskOperationError defined in Table 5.5.2-8 instead of the fields in Table 5.5.2-4b.

When a Network AI Agent receives a task/delete request from another Network AI Agent, the response message after a successful operation contains the fields presented in Table 5.5.2-4c.

Table 5.5.2-4c: Fields of the TaskDeleteResult data type

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the deleted taskID identifier. This shall be the same taskIDContent value from the corresponding task/delete request message. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.

A "task/delete" request may fail due to invalid fields in the request, syntax/type errors, lacking authorization, and internal errors of the receiving agent. Each of these errors lead to failed operation, triggering a TaskOperationError defined in Table 5.5.2-8 instead of the fields in Table 5.5.2-4c.

Table 5.5.2-5a: Fields of a task/subscribe request

Field Name	Type	Presence	Description
agentIDContent	string[1..1]	M	Value of the agentID identifier. This forms the composite AgentID field together with the AgentIDFormat field. This field shall not be NULL or empty. The composite Agent ID is used by the receiving agent to evaluate whether the requesting agent is authorized the receive updates about the subscribed task. The authorization policy may depend on operator's configuration and is out of the scope of the present document.
agentIDFormat	string[1..1]	M	The format or namespace of the AgentIDContent field (e.g. UUID-v4, operator-assigned). This forms the composite AgentID field together with the AgentIDContent field. This field shall not be NULL or empty.
taskIDContent	string[1..1]	M	Value of the taskID identifier that the sender agent requests to receive notifications about. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.
notifyOnState	TaskState[0..*]	O	The specific TaskState(s) that, when transitioned into, will trigger a notification message to the subscriber. The list of possible TaskStates is defined in Table 5.5.2-7. If this field is not present, then any state transition shall trigger a notification message to the subscriber.
subscriptionExpiry	string[0..1]	O	ISO 8601 [14] UTC datetime after which the subscription shall be automatically cancelled by the receiving agent. If absent, the subscription persists until explicitly cancelled via task/unsubscribe.

Table 5.5.2-5b: Fields of the TaskSubscribeResult data type

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the taskID identifier of the subscribed task. This shall be the same taskIDContent value from the corresponding task/subscribe request message. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.

Table 5.5.2-6a: Fields of a task/unsubscribe request

Field Name	Type	Presence	Description
agentIDContent	string[1..1]	M	Value of the agentID identifier. This forms the composite AgentID field together with the AgentIDFormat field. This field shall not be NULL or empty. The composite Agent ID is used by the receiving agent to evaluate whether the requesting agent is authorized the receive updates about the subscribed task. The authorization policy may depend on operator's configuration and is out of the scope of the present document.
agentIDFormat	string[1..1]	M	The format or namespace of the AgentIDContent field. This forms the composite AgentID field together with the AgentIDContent field. This field shall not be NULL or empty.
taskIDContent	string[1..1]	M	Value of the taskID identifier that the sender agent requests to stop receiving notifications about. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.

Table 5.5.2-6b: Fields of the TaskUnsubscribeResult data type

Field Name	Type	Presence	Description
taskIDContent	string[1..1]	M	Value of the taskID identifier of the unsubscribed task. This shall be the same taskIDContent value from the corresponding task/subscribe request message. This forms the composite taskID field together with the taskIDFormat field. This field shall not be NULL or empty.
taskIDFormat	string[1..1]	M	Format or namespace of the taskIDContent field. This forms the composite taskID field together with the taskIDContent field. This field shall not be NULL or empty.
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.

Table 5.5.2-7: Enumeration of TaskState values

Value	Description
"unknown"	The task is in an unknown or indeterminate state.
"submitted"	Indicates that a task has been successfully submitted and acknowledged.
"working"	Indicates that a task is actively being processed by the agent.
"completed"	Indicates that a task has finished successfully. This is a terminal state.
"failed"	Indicates that a task has finished with an error. This is a terminal state.
"timeout"	The task finished with a timeout. This is a terminal state.
"cancelled"	Indicates that a task was cancelled before completion. This is a terminal state.
"inputRequired"	Indicates that the agent requires additional user input to proceed. This is an interrupted state.
"authenticationRequired"	Indicates that authentication is required to proceed. This is an interrupted state.

Table 5.5.2-8: Fields of the TaskOperationError data type

Field Name	Type	Presence	Description
messageID	string[1..1]	M	Per-request correlation identifier. The responding agent shall echo this value unchanged in the response message.
taskIDContent	string[0..1]	CM	Value of the composite identifier of the task from the request. This field is present only if the task identified in the request has been found in the responding Network AI Agent's database.
taskIDFormat	string[0..1]	CM	Format of the composite identifier of the task from the request. This field is present only if the task identified in the request has been found in the responding Network AI Agent's database.
errorCode	string[1..1]	M	Machine-readable error code.
errorMessage	string[1..1]	M	Human-readable error description.

5.5.3 Protocol

The options listed in clause 5.3.3 are valid for the interface between multiple Network AI Agents.

The protocol binding options listed in clause 5.3.3 are valid for the interface between Network AI Agents. When JSON-RPC 2.0 is used, the method names defined in Table 5.5.2-1 shall be encoded as the "method" member of the JSON-RPC 2.0 request object (see [3]). The parameters defined in Table 5.5.2-2 through Table 5.5.2-8 shall be encoded in the "params" member.

5.5.4 Security

5.5.4.1 Authentication

Authentication of Network AI Agents accessing the interface defined in clause 5.5 of the present document shall comply with clause 5.8.2.4 of the present document. Each Network AI Agent participating in agent-to-agent communication shall authenticate using mutual TLS (mTLS) with a valid client certificate issued or validated by an operator-trusted certificate authority, consistent with 3GPP TS 33.501 [5]. A Network AI Agent shall reject any connection from a peer agent that fails mTLS authentication.

NOTE: The agent-to-agent interface defined in clause 5.5 is symmetric; either agent is able to act as initiator. Both the initiating and receiving agents are able to authenticate each other via mTLS regardless of which agent initiates the connection.

5.5.4.2 Authorization

Authorization of Network AI Agents shall comply with clause 5.8.3.3 of the present document. A Network AI Agent shall verify that the requesting agent is authorized to invoke the requested method prior to processing any request. Authorization shall be based on the authenticated agent identity established during mTLS and the agent's registered profile in the ARF reflected by its composite AgentID, as specified in clause 5.4.

The following authorization requirements apply per method:

1) Task/create: requires that the requesting agent holds a valid authorization to delegate tasks to the receiving agent, as established by operator policy. The receiving agent shall verify that the composite AgentID carried in the request is authorized to assign a new task to itself.

2) Task/update: requires that the requesting agent identified by the composite AgentID is either the original creator of the task, or an operator-authorized orchestrating agent. The receiving agent shall reject update requests, including cancellation requests, from agents not meeting these criteria.

3) Task/get: requires that the requesting agent is either the creator of the task, a registered subscriber to that task or an operator-authorized agent. The receiving agent shall return only information pertaining to tasks for which the requesting agent holds a valid authorization.

4) Task/delete: requires that the requesting agent is the original creator of the task identified by the composite AgentID, or an operator-authorized orchestrating agent. The receiving agent shall reject deletion requests from agents not meeting these criteria.

5) Task/subscribe: requires that the requesting agent, identified by the composite AgentID in the subscription request (Table 5.5.2-5a) is an authorized participant to retrieve information about the specified task. The receiving agent shall perform validation before accepting the subscription.

6) Task/unsubscribe: requires that the requesting agent, identified by the composite AgentID in the subscription request (Table 5.5.2-6a) is an authorized participant to retrieve information about the specified task. The receiving agent shall perform validation before accepting the subscription.

7) Task/notify: shall only be issued by the agent that owns the task (i.e. the agent that accepted the original task/create request). Subscriber agents shall reject task/notify messages received from agents that are not the task owner.

5.5.4.3 Data Access Constraints

A Network AI Agent shall not include Class 3 or Class 4 data as defined in clause 5.8.3.5 of the present document in agent-to-agent message payload unless the receiving agent has explicit operator authorization to access that data class. Task result payloads returned via task/get or delivered via task/notify shall be filtered to exclude data not pertaining to the receiving agent's participation in the service.

5.5.4.4 Transport Security

Transport security for the agent-to-agent interface shall comply with clause 5.8.4 of the present document. All agent-to-agent interface interactions shall be carried over TLS 1.3 or later. The preferred protocol binding is HTTP/3+JSON-RPC 2.0 as specified in clause 5.3.3. If HTTP/3 is not supported by the deployment environment, HTTP/2+JSON-RPC 2.0 shall be used. HTTP/1.1 bindings shall not be used for agent-to-agent communication.

5.6 Interface between Network AI Agent and Resources

5.6.1 Interface between Network AI Agent and Data

5.6.1.1 Requirements

In a mobile network, the architecture shall comprise network elements that are intended for network data storage. Depending on the task-at-hand, Network AI Agents shall have access to all relevant data to improve their decision-making as well as to complete the necessary sub-tasks that rely on the existing knowledge (e.g. UE subscription information and UE policy information). This calls for a new interface between a Network AI Agent and a knowledge base (i.e. a data server) that exposes these data to agents.

In contrast to the existing service-based architecture, where data provisioning is entirely specified and pre-defined, limiting the extensibility and flexibility of data management in an AI-agent based mobile network architecture, this can be done by relying on inferencing capabilities of agents during operation. In other words, AI agents do not need to be implemented as rigid functional blocks that can only read from or write to a data storage, based on a standardized and pre-defined set of data types, as in the case of pre-defined analytics IDs for Network Data Analytics Function (NWDAF) in service-based architecture. On the contrary, they shall be able to discover the available data resources, as in MCP, understand the description of each data entry, and subsequently identify which data is required for the completion of the corresponding task. This is then followed by requesting the corresponding data entry from the knowledge base. Moreover, the interface shall allow the agents to store new data (e.g. measurement data, operational logs, and context data) so that it becomes accessible for future tasks that would benefit from the availability of these data (e.g. by the same agent or other AI agents). As such a knowledge base is shared by multiple entities, it shall implement certain security mechanisms to prevent unauthorized access.

5.6.1.2 Methods and Data Objects

The methods for the agent-to-data interface are given in Table 5.6.1.2-1.

Table 5.6.1.2-1: Methods for the agent-to-data interface

Method Name	Description	Inputs	Outputs
data/list	The method to indicate that the Network AI Agent (i.e. client) requests to query the list of available data resources (exposed by the server).	params: ResourceReqParams[0..1] (see Table 5.6.1.2-2)	On success: • HTTP Status: 200 OK. • The HTTP body contains an array of DataInfo objects corresponding to the data resources available at the server (see Table 5.6.1.2-3). On failure: • JSONRPCError.
data/read	The method to indicate that the Network AI Agent (i.e. client) requests to read a specific data resource available at the data server. This typically follows a data discovery request (with method as "data/list").	uri: string[1..1] The URI of the data resource to retrieve, as obtained from a prior data/list response. Shall not be NULL or empty. Format per [7].	On success: • HTTP status: 200 OK. • HTTP body contains one or more Part[1..*] objects constituting the data content, which is defined in Table 5.6.1.2-4. On failure: • JSONRPCError.
data/create	A Network AI Agent requests to create a new data resource at the data server. Shall fail if a resource with the same name already exists in the server's namespace.	params: DataCreateParams[1..1] (see Table 5.6.1.2-5)	On success: • HTTP Status 201 Created. HTTP body contains a JSON object with a single field uri: string[1..1], the server-assigned URI of the newly created resource. On failure: • JSONRPCError.
data/update	A Network AI Agent requests to update the content or metadata of an existing data resource. Shall fail if no resource with the specified URI exists at the data server.	params: DataUpdateParams[1..1] (see Table 5.6.1.2-5a)	On success: • HTTP Status 200 OK. On failure: • JSONRPCError.
data/delete	A Network AI Agent requests to delete an existing data resource and all associated metadata. Shall fail if no resource with the specified URI exists. See note below.	uri: string[1..1] The URI of the data resource to delete. Shall not be NULL or empty.	On success: • HTTP Status 200 OK. On failure: • JSONRPCError.
data/subscribe	The method indicates that the Network AI Agent requests to subscribe to data resource-related modifications. If ResourceReqParams is included, then the type-filter and/or semantic-filter is used to identify the notification triggering condition. If the request contains a URI key, then the subscription applies to the specified resource with the respective URI. If the ResourceReqParams does not contain any filter information, then the subscription applies for any change.	params: DataSubscribeParams[1..1] (see Table 5.6.1.2-6a)	On success: • HTTP Status: 201 Created. • HTTP body contains a subscriptionID: string[1..1] assigned by the data server, which the subscribing agent shall use to cancel the subscription via data/unsubscribe. On failure: • JSONRPCError.

Method Name	Description	Inputs	Outputs
data/unsubscribe	A Network AI Agent requests to cancel an existing subscription to data resource change events.	dataUnsubscribeParams: DataUnsubscribeParams[1..1] (see Table 5.6.1.2-6)	On success: • HTTP Status 200 OK. On failure: • JSONRPCError.
data/notify	The method indicates a notification message that is sent by the data server to all subscribers (i.e. Network AI Agent) when there is a change event in the subscribed data.	dataNotifyParams: DataNotifyParams[1..1] (see Table 5.6.1.2-7)	NA

A data server shall return a JSONRPC error if the method in the request message does not match any of the specified methods in Table 5.6.1.2-1. If JSON-RPC 2.0 is used, the "code" and "message" fields within the "error" object shall have the value -32601 and "method not found", respectively.

A data server shall return a JSON-RPC error if the agent sends a "data/read", a "data/update", or a "data/delete" request and a resource associated with the same name cannot be found in the server's namespace. If JSON-RPC 2.0 is used, then the "code" and "message" fields within the "error" object shall have the value -32602 and "Invalid params", respectively, and the "data" object shall not be omitted and shall contain a descriptive error string, e.g. "reason": "no resource found".

A data server shall return a JSON-RPC error if the agent sends a "data/create" and a resource with the same name already exists in the server's namespace. If JSON-RPC 2.0 is used, then the "code" and "message" fields within the "error" object shall have the value -32602 and "Invalid params", respectively, and the "data" object shall not be omitted and shall contain a descriptive error string, e.g. "reason": "resource already exists".

A data server may reject "data/delete" requests based on data retention policy or operator-configured access controls, regardless of whether the resource URI is valid. The data server shall return a JSONRPCError with a descriptive error message in such cases. If JSON-RPC 2.0 is used and request is rejected due to data retention policy or operator-configured access controls, then the response shall indicate one of the implementation-defined server-errors, i.e. from 32000 to -32099, and an implementation-specific descriptive error message.

The data objects used for the agent-to-data interface interaction are shown in Table 5.6.1.2-2 to Table 5.6.1.2-7.

Table 5.6.1.2-2: Fields of the ResourceReqParams object

Field Name	Type	Presence	Description
typeFilter	string[0..*]	O	Filter by data resource type. If present, the data server shall return only resources whose type matches one of the specified values. Valid enumeration values are: "operatorData", "logData", "visualData", "textData", "rawData". Shall contain at least one value if present.
semanticFilter	string[0..*]	O	Filter by semantic description of data resources. If present, the data server shall include only resources whose description matches the specified keywords or semantic criteria. See note.
metadata	map(string, DataValue)[0..1]	O	Arbitrary key-value matching associated with the request.
NOTE: The matching algorithm (e.g. keyword matching, vector similarity) is FFS.			

Table 5.6.1.2-3: Fields of the DataInfo object

Field Name	Type	Presence	Description
uri	string[1..1]	M	Unique resource identifier assigned by the data server at creation time. Format per [7]. This URI shall be used in subsequent data/read, data/update, data/delete, and data/subscribe requests to reference this resource. Shall not be NULL or empty.
type	enum[1..1]	M	Type of the data resource. Valid quoted enumeration values: "operatorData", "logData", "visualData", "textData", "rawData". See Table 5.6.1.2-2.
name	string[0..1]	O	Human-assigned name of the data resource, as provided in the DataCreateParams object. If absent, the data server shall not substitute a default value.
description	string[1..1]	M	Human-readable description of the data resource, as provided in the DataCreateParams object. Shall not be NULL or empty.
sourceID	string[0..1]	O	Network identifier of the data provider that created this resource, as provided in the DataCreateParams object. Format is implementation-specific.
createdAt	string[1..1]	M	ISO 8601 [14] UTC datetime at which the data resource was created. Shall include an explicit UTC offset (Z or +00:00).
modifiedAt	string[0..1]	O	ISO 8601 [14] UTC datetime at which the data resource was most recently modified via data/update. If absent, the resource has not been modified since creation.
mimeType	string[0..1]	O	Optional MIME type of the data resource. The value of this field shall be compliant with IETF RFC 2046 [10].
metadata	map(string, DataValue)[0..1]	O	Key-value metadata associated with the data resource, as provided in the DataCreateParams or most recent DataUpdateParams object.
NOTE: The versioning and schema evolution mechanism for DataInfo objects is FFS. A candidate approach is to align with the CapabilityVersion design in Table 5.2.2-5.			

Table 5.6.1.2-4: Part object used for structured representation of the data content

Field Name	Type	Presence	Description
text	string[0..1]	CM	The string content of a textual data resource. It is mandatory if the data object has text type. Shall be present when the data resource type is "textData". Shall be absent otherwise.
raw	bytes[0..1]	CM	The raw byte content of a data resource. It is mandatory if the data object has a binary type (e.g. a data file). In JSON serialization this field shall contain a Base64-encoded representation of the binary data. Shall be present when the data resource type is binary. Shall be absent otherwise.
mimeType	string[0..1]	O	The media type of the part content, such as "text/plain", "application/json", "image/png", "application/octet-stream". See [10]. If absent, the consumer shall interpret the content using the MIME type of the enclosing data resource, if provided, or fall back to operator-defined defaults.
NOTE: Part object contains one of "text" or "raw" fields.			

Table 5.6.1.2-5: Fields of the DataCreateParams data type

Field Name	Type	Presence	Description
type	enum[1..1]	M	Type of the data resource to create. Valid values: "operatorData", "logData", "visualData", "textData", "rawData". See Table 5.6.1.2-2 for the definition of the type enumeration.
name	string[1..1]	M	Name of the data resource to create. The data server shall reject requests where this field is NULL or empty.
description	string[1..1]	M	Human-readable description of the data resource. The data server shall reject requests where this field is NULL or empty.
parts	Part[1..*]	M	One or more Part objects constituting the content of the data resource. See Table 5.6.1.2-4.
sourceID	string[0..1]	O	Network identifier of the data provider. Format is implementation-specific.
metadata	map(string, DataValue)[0..1]	O	Arbitrary key-value metadata associated with the data resource.

Table 5.6.1.2-5a: Fields of the DataUpdateParams data type

Field Name	Type	Presence	Description
uri	string[1..1]	M	URI of the existing data resource to update, as previously assigned by the data server. Shall not be NULL or empty. Format per [7].
description	string[0..1]	O	Updated human-readable description of the data resource. If absent, the existing description is retained unchanged.
parts	Part[1..*]	CM	Updated content of the data resource. Shall be present when the update modifies the resource content. If absent, only the metadata shall be updated.
metadata	map(string, DataValue)[0..1]	O	Updated key-value metadata. If present, the data server shall replace the existing metadata in full. If absent, the existing metadata is retained unchanged.
sourceID	string[0..1]	O	Network identifier of the data provider performing the update.
NOTE: A data/update request that carries neither parts nor metadata is valid but has no effect. The data server treats such requests as a no-op and return HTTP Status 200 OK.			

Table 5.6.1.2-6: Fields of the DataUnsubscribeParams data type

Field Name	Type	Presence	Description
subscriptionID	string[1..1]	M	Identifier of the subscription to cancel, as assigned by the data server at the time the data/subscribe request was accepted. Shall not be NULL or empty.

Table 5.6.1.2-6a: Fields of the DataSubscribeParams data type

Field Name	Type	Presence	Description
resourceReq	ResourceReqParams[0..1]	O	Optional resource-level filters used to determine which changes trigger notifications.
uri	string[0..1]	O	URI of a specific data resource. If present, the subscription applies only to that resource.

Table 5.6.1.2-7: Fields of the DataNotifyParams data type

Field Name	Type	Presence	Description
subscriptionID	string[1..1]	M	Identifier of the subscription that triggered this notification. Shall not be NULL or empty.
changeType	enum[1..1]	M	Type of change event. Enumeration: "created", "modified", "deleted".
uri	string[0..1]	C	URI of the affected data resource. Shall be present when the notification was triggered by a resource-specific subscription.
dataInfo	DataInfo[0..1]	O	Updated DataInfo object for the affected resource. Present when changeType is "created" or "modified". Absent when changeType is "deleted".

5.6.1.3 Protocol

The Network AI Agent to data server interface shall use JSON-RPC 2.0 serialization over HTTP/2 or HTTP/3. The method names defined in Table 5.6.1.2-1 shall be encoded as the "method" member of the JSON-RPC 2.0 request object. The parameters defined in Table 5.6.1.2-2 through Table 5.6.1.2-7 shall be encoded in the "params" member. The fields of the response messages shall be encoded in "result" member. Transport security requirements, including TLS 1.3, are specified in clause 5.6.3.4.

5.6.2 Interface between Network AI Agent and Tools

5.6.2.1 Requirements

The interface between Network AI Agents and external tools shall allow an agent to discover the tools available at external tool server(s), to invoke tools by providing the required input parameters, and to receive tool call results. The interface shall support dynamic addition, modification, and deletion of tools. The interface shall also support event-based interaction between agents and external tool servers, enabling agents to subscribe to tool modification notifications and enabling servers to notify agents when tool metadata changes. A publish-subscribe pattern shall be used between agents and servers, where agents subscribe to receive any modification of tool information and the servers notify the agents when such an event occurs.

5.6.2.2 Methods and Data Objects

The methods for the agent-to-tool interface are given in Table 5.6.2.2-1.

Table 5.6.2.2-1: Overview of agent-to-tool interface methods

Method Name	Description	Inputs	Outputs
tools/list	A Network AI Agent requests to query available tool information from the tool server, optionally filtered by ToolListFilter.	toolListFilter: ToolListFilter[0..1] (see Table 5.6.2.2-2a)	On success: • HTTP Status: 200 OK. • HTTP body contains a "tools" element comprised of ToolInfo-typed objects as defined in Table 5.6.2.2-2. An empty list indicates that no tools matching the request criteria are currently available. On failure: • JSONRPCError.
tools/call	A network agent requests to invoke a tool exposed by the external tool server.	params: ToolCallParams[1..*] (see Table 5.6.2.2-3)	On success: • HTTP Status: 200 OK • HTTP body contains a "result" element, which shall contain one or more Part objects (see Table 5.6.1-4) representing the tool output. On failure: • JSONRPCError.
tools/subscribe	A network agent requests to subscribes to modifications in a particular tool.	params: ToolSubscriptionParams[1..1] (see Table 5.6.2.2-4)	On success: • HTTP Status: 201 Created. • HTTP body contains subscriptionID : string[1..1], assigned by the tool server, which the subscribing agent shall use to cancel the subscription via tools/unsubscribe. On failure: • JSONRPCError.
tools/unsubscribe	A network agent requests to cancel an existing subscription to tool information change events.	params: ToolUnsubscribeParams[1..1] (see Table 5.6.2.2-5)	On success: • HTTP Status: 200 OK. On failure: • JSONRPCError.
tools/notify	The notification message sent by the tool server to a network agent to provide the most recent tool information.	params: ToolNotifyParams[1..1] (see Table 5.6.2.2-6)	NA

A tool server shall return a JSONRPC error if the method in the request message does not match any of the specified methods in Table 5.6.2.2-1. If JSON-RPC 2.0 is used, the "code" and "message" fields within the "error" object shall have the value -32601 and "method not found", respectively.

A tool server shall return a JSON-RPC error if the agent sends a "tools/call" or a "tools/subscribe" request and a tool associated with the name contained in the ToolCallParams or ToolSubscriptionParams object cannot be found in the server's namespace. If JSON-RPC 2.0 is used, then the "code" and "message" fields within the "error" object shall have the value -32602 and "Invalid params", respectively, and the "data" object shall not be omitted and shall contain a descriptive error string, e.g. "reason": "no tool found".

A tool server shall return a JSON-RPC error if the agent sends a "tools/call" request and the arguments provided in the request message do not conform to the tool's input schema. If JSON-RPC 2.0 is used, then the "code" and "message" fields within the "error" object shall have the value -32602 and "Invalid params", respectively, and the "data" object shall not be omitted and shall contain a descriptive error string, e.g. "reason": "unexpected argument", "reason": "argument type mismatch".

A tool server shall return a JSON-RPC error if the agent sends a "tools/unsubscribe" request and a subscription associated with the subscriptionID contained in the ToolUnsubscribeParams object cannot be found in the server's namespace. If JSON-RPC 2.0 is used, then the "code" and "message" fields within the "error" object shall have the value -32602 and "Invalid params", respectively, and the "data" object shall not be omitted and shall contain a descriptive error string, e.g. "reason": "no subscription found".

When a Network AI Agent sends a tool discovery request to a tool server (by specifying "method": "tools/list"), the response message contains information about the available tools that are exposed to the agent. This is done by providing a ToolInfo object per each available tool at the tool server. Table 5.6.2.2-2 lists the fields in ToolInfo object that is contained in the tool discovery response. The tool discovery request may contain a filter of type ToolListFilter defined in Table 5.6.2.2-2a, which is used when determining the content of the response message. If present, then tool filter allows the tool repository to identify a subset of tools that match the tool filter. If all tools satisfy the specified criterion, then the response message has no effect on the tool discovery response. The type of the tool filter may determine entity that applies the filtering mechanism. As an example, if "semantic" filtering is indicated, where the client provides the description of the tools of interest, then inference mechanism may be used to select the relevant tools matching the description. If "keywords" is used, then a rule-based mechanism may be employed to identify the tools that have the specified keywords in ToolInfo object defined in Table 5.6.2.2-2.

Table 5.6.2.2-2: Definition of ToolInfo object

Field Name	Type	Presence	Description
name	string[1..1]	M	Name of the tool.
description	string[1..1]	M	Description of the tool in natural language.
inputSchema	SchemaObject[1..1] (see Table 5.6.2.2-7)	M	Schema defining the required and optional input parameters of the tool. The tool server shall reject tool/call requests whose arguments do not conform to this schema.
outputSchema	SchemaObject[0..1] (see Table 5.6.2.2-7)	O	Object defining the structure of tool's output, e.g. key-value pairs, where a value may be of different types, e.g. string, number, and object.
examples	string[0..*]	O	List of examples of tool utilization. Examples can help a tool client use the tool correctly (e.g. for in-context learning).
metadata	map(string, DataValue)[0..1]	O	Key-value pairs for optional metadata information relevant for the tool.

Table 5.6.2.2-2a: Definition of ToolListFilter object

Field Name	Type	Presence	Description
toolFilterContent	string[1..1]	M	Value of type string containing tool filter information that may reduce the returned tool information.
toolFilterType	enum[1…1]	M	Type of the filtering mechanism to-be-applied. Enumeration: "semantic", "keywords". The value of toolFilterType may be used for: • determining the entity that applies the indicated tool filter; • how the value of the toolFilterContent field is interpreted. See note.
NOTE: The extension of toolFilterType enumeration beyond "semantic" and "keywords" is FFS.			

Table 5.6.2.2-3 presents the ToolCallParams object that is a mandatory input for a tool invocation request.

Table 5.6.2.2-3: Definition of ToolCallParams object

Field Name	Type	Presence	Description
name	string[1..1]	M	Name of the tool to-be-called.
arguments	map(string, DataValue)[0..1]	CM	Input arguments to the tool call, as required by inputSchema from ToolInfo object. If tool does not have any mandatory input, then this field is not included in the tool call request.

The tool server receiving a tool invocation request shall validate the arguments field in the request against the inputSchema associated with the ToolInfo of the corresponding tool identified based on the name field. If the arguments do not conform to the schema, then the tool server shall respond with a JSON-RPC error message indicating that the input arguments are invalid. If the JSON-RPC 2.0 is used, then this corresponds to error code with value -32602. The exact message sent together with the error code (i.e. "message" key) explaining the reason is specific to the tool server's implementation.

A tool client may request to be notified about any modifications in the tool information that it has already discovered previously. This can be done by sending a "tools/subscribe" request to the tool server. The subscription request contains the ToolSubscriptionParams object, as given in Table 5.6.2.2-4.

Table 5.6.2.2-4: Fields of the ToolSubscriptionParams data type

Field Name	Type	Presence	Description
toolName	string[0..1]	O	Name of the specific tool to subscribe to. If absent, the subscription applies to any tool change event at the tool server. If present, the tool server shall reject the subscription if no tool with the specified name is currently registered.
subscriptionExpiry	string[0..1]	O	ISO 8601 [14] UTC datetime after which the subscription shall be automatically cancelled by the tool server. If absent, the subscription persists until explicitly cancelled by the subscribing agent via tools/unsubscribe.

A subscriber may maintain multiple subscriptions that match the same tool change event (e.g. a global subscription and a specific toolName subscription). In such cases, duplicate notifications may be delivered; de-duplication behaviour is implementation-specific and outside the scope of the present document.

Table 5.6.2.2-5 presents the ToolUnsubscribeParams object, which is a mandatory input parameter when a Network AI Agent requests the cancellation of an active subscription. The behaviour of the subscription cancellation depends on the parameters of the original subscription creation request:

- If the subscription associated with the subscriptionID was created via a "tools/subscribe" request that included a specific toolName, receipt of the unsubscribe message shall terminate all subsequent change notifications related to that specific tool for the designated subscriber. Other active subscriptions for the subscriber shall remain unaffected.
- If the subscription associated with the subscriptionID was created via a tools/subscribe request that did not specify a toolName, receipt of the tools/unsubscribe message shall terminate all subsequent notifications bound to that specific subscriptionID.
- The termination of a subscription shall not impact, terminate, or suppress notifications from any subscriptions created via independent tools/subscribe requests yielding distinct subscriptionIDs.

Table 5.6.2.2-5: Fields of the ToolUnsubscribeParams data type

Field Name	Type	Presence	Description
subscriptionID	string[1..1]	M	Identifier of the subscription to cancel, as assigned by the tool server when the tools/subscribe request was accepted. Shall not be NULL or empty.

Table 5.6.2.2-6: Fields of the ToolNotifyParams data type

Field Name	Type	Presence	Description
subscriptionID	string[1..1]	M	Identifier of the subscription that triggered this notification. Shall not be NULL or empty.
changeType	enum[1..1]	M	Type of change event. Enumeration: "added", "modified", "removed".
toolInfo	ToolInfo[0..1]	C	Updated ToolInfo object. Shall be present when changeType is "added" or "modified". Shall be absent when changeType is "removed".

Table 5.6.2.2-7: Definition of the SchemaObject object

Field Name	Type	Presence	Description
type	string[1..1]	M	Type of the schema object with the literal value "object" (i.e. "type": "object").
properties	map(string, SchemaPropertyObject)[0..1] (see Table 5.6.2.2-8)	O	A dictionary mapping argument names to their respective data types and schemas. This field defines all available parameters that the tool can accept. If a key is listed in the "required" field, it shall be present here. If the tool does not allow any input argument, then this field shall be omitted.
required	string[0…*]	O	List of required (i.e. mandatory) input arguments for the tool call. Every item shall exactly match a key in the properties object.

Table 5.6.2.2-8: Definition of SchemaPropertyObject object

Field Name	Type	Presence	Description
type	enum[1..1]	M	Type of the input argument. Valid values are: "string", "boolean", "number", "object", "integer", and "array".
description	string[0..1]	O	Description of the input argument.
minimum	number[0..1]	O	Minimum numerical value accepted for the argument. This field is present only if the argument is of type "number" or "integer".
maximum	number[0..1]	O	Maximum numerical value accepted for the argument. This field is present only if the argument is of type "number" or "integer".
minLength	number[0..1]	O	Minimum accepted length of the string instance defined as the number of its characters. This value is present only if the input argument is of type "string". Any string typed input argument is valid only if its length is greater than, or equal to the value of this keyword. Omitting this keyword has the same behaviour as value of 0. Value of this keyword shall be a non-negative integer.
maxLength	number[0..1]	O	Maximum accepted length of the string instance defined as the number of its characters. This value is present only if the input argument is of type "string". Any string typed input argument is valid only if its length is less than, or equal to the value of this keyword. Value of this keyword shall be a non-negative integer.
minItems	number[0..1]	O	Minimum number of items that an array typed input argument shall contain. This value is present only if the input argument is of type "array". If the input array has fewer items than the specified value, validation against this keyword fails. Value of this keyword shall be a non-negative integer. Omitting this keyword has the same behaviour as a value of 0.
maxItems	number[0..1]	O	Maximum number of items that an array typed input argument shall contain. This value is present only if the input argument is of type "array". If the input array has more items than the specified value, validation against this keyword fails. Value of this keyword shall be a non-negative integer.

Field Name	Type	Presence	Description
default	any[0..1]	O	The default value of the input argument. If the argument type is string, then the value of this keyword shall be a valid string as defined in IETF RFC 8259 [12]. If the argument type is boolean, then this can be either true or false. If the argument type is of type object, then the value of this keyword shall be a valid JSON object as defined in IETF RFC 8259 [12]. If the argument type is a number, then the value of this keyword shall be a valid numeric as defined in IETF RFC 8259 [12]. If the argument type is an array, then the value of this keyword shall be a valid array as defined in IETF RFC 8259 [12].

NOTE: The semantics of the "default" keyword follow JSON Schema and IETF RFC 8259 [12]; the value is not required to be used by tool servers or clients.

5.6.2.3 Protocol

The Network AI Agent to tool interface shall use JSON-RPC 2.0 serialization over HTTP/2 or HTTP/3. The method names defined in Table 5.6.2.2-1 shall be encoded as the "method" member of the JSON-RPC 2.0 request object. The parameters defined in Table 5.6.2.2-2 through Table 5.6.2.2-8 shall be encoded in the "params" member. The fields of the response messages shall be encoded in "result" member. Transport security requirements, including TLS 1.3, are specified in clause 5.6.3.4.

5.6.3 Security

5.6.3.1 Authentication

Authentication of Network AI Agents accessing the interfaces defined in clause 5.6 of the present document shall comply with clause 5.8.2.4 of the present document and 3GPP TS 33.501 [5]. Each Network AI Agent accessing a data server or tool server shall authenticate using mutual TLS (mTLS) with a valid client certificate issued or validated by an operator-trusted certificate authority. A data server or tool server shall reject any connection from a Network AI Agent that fails mTLS authentication.

NOTE: The data server and tool server are asymmetric interfaces. The Network AI Agent acts as the client and the data server or tool server acts as the server. The server-side entity presents a valid TLS server certificate. The agent verifies this certificate against the operator-trusted certificate authority before transmitting any request.

5.6.3.2 Authorization

5.6.3.2.0 Introduction

Authorization of Network AI Agents shall comply with clause 5.8.3.3 of the present document. A data server or tool server shall verify that the requesting agent is authorized to invoke the requested method prior to processing any request. Authorization shall be based on the authenticated agent identity established during mTLS and the agent's registered profile in the ARF, as specified in clause 5.4.

The following authorization requirements apply per interface.

5.6.3.2.1 Agent-to-Data Interface

Table 5.6.3.2.1-1 defines the authorization requirements for the eight methods of the agent-to-data interface.

Table 5.6.3.2.1-1: Authorization requirements for the methods of the agent-to-data interface

Method Name	Requirement
data/list	Requires that the requesting agent holds a valid read authorization for the data server. The data server shall filter the returned DataInfo list to include only resources for which the requesting agent holds at least read authorization. A data/list response shall not disclose the existence of resources to which the requesting agent has no access rights.
data/read	Requires that the requesting agent holds read authorization for the specific data resource identified by the uri. The data server shall reject requests for resources for which the requesting agent does not hold read authorization, and shall return a JSONRPCError with an access-denied error code. The data server shall not confirm or deny the existence of a resource for which the requesting agent has no access rights.
data/create	Requires that the requesting agent holds write authorization for the data server. The data server shall additionally enforce operator-configured constraints on the data types and maximum resource size that a given agent is permitted to create.
data/update	Requires that the requesting agent holds write authorization for the specific data resource identified by the uri. The data server shall reject update requests from agents that did not create the resource unless the agent holds explicit operator-granted update authorization for that resource.
data/delete	Requires that the requesting agent holds delete authorization for the specific data resource identified by the uri. The data server shall reject deletion requests from agents that did not create the resource unless the agent holds explicit operator-granted delete authorization for that resource. Operator-configured data retention policies shall take precedence over agent-initiated deletion requests.
data/subscribe	Requires that the requesting agent holds read authorization for the resource or resource type it is subscribing to. The data server shall validate whether the requesting agent is authorized to gain read access for the requested resource or resource type prior to accepting the subscription.
data/unsubscribe	Requires that the requesting agent is the agent that created the subscription identified by the subscriptionID. The data server shall reject cancellation requests from agents that did not originate the subscription.
data/notify	Shall only be issued by the data server. A Network AI Agent receiving a data/notify message shall verify that the message originates from the data server that accepted the original data/subscribe request, using the server's mTLS identity.

5.6.3.2.2 Agent-to-Tool Interface

Table 5.6.3.2.2-1 defines the authorization requirements for the five methods of the agent-to-tool interface.

Table 5.6.3.2.2-1: Authorization requirements for the methods of the agent-to-tool interface

Method Name	Requirement
tools/list	Requires that the requesting agent holds a valid authorization to query the tool server. The tool server shall filter the returned ToolInfo list to include only tools for which the requesting agent holds at least invocation authorization. A tools/list response shall not disclose the existence of tools to which the requesting agent has no access rights.
tools/call	Requires that the requesting agent holds invocation authorization for the specific tool identified by the toolName field of the ToolCallParams object. The tool server shall validate the arguments field against the tool's inputSchema prior to invocation and shall reject requests whose arguments do not conform. The tool server shall additionally support enforcement of operator-configured rate limits on tool invocations per agent.
tools/subscribe	Requires that the requesting agent holds at least read authorization for the information associated with the tool(s). The tool server shall validate the agent's authorization prior to accepting the subscription.
tools/unsubscribe	Requires that the requesting agent is the agent that created the subscription identified by the subscriptionID. The tool server shall reject cancellation requests from agents that did not originate the subscription.
tools/notify	Shall only be issued by the tool server. A Network AI Agent receiving a tools/notify message shall verify that the message originates from the tool server that accepted the original tools/subscribe request, using the server's mTLS identity.

5.6.3.3 Data Access Constraints

A data server shall classify all stored data resources according to the data classification scheme defined in clause 5.8.3.5 of the present document. A Network AI Agent shall not be authorized to read, create, update, or delete Class 3 or Class 4 data resources unless explicitly authorized by operator policy. The data server shall enforce these constraints at the resource level, not solely at the interface level.

A tool server shall not return tool outputs containing Class 3 or Class 4 data in a tools/call response to an agent that does not hold the corresponding data class authorization. Where a tool result would otherwise contain such data, the tool server shall either redact the data or reject the invocation with a JSONRPCError indicating insufficient authorization.

The sourceID field in DataCreateParams and DataUpdateParams shall be validated by the data server against the authenticated agent identity. A data server shall reject requests where the sourceID does not match the authenticated mTLS identity of the requesting agent.

5.6.3.4 Transport Security

Transport security for the agent-to-data and agent-to-tool interfaces shall comply with clause 5.8.4 of the present document. All interactions on these interfaces shall be carried over TLS 1.3 or later. The preferred protocol binding is HTTP/3+JSON-RPC 2.0 as specified in clauses 5.6.1.3 and 5.6.2.3. If HTTP/3 is not supported by the deployment environment, HTTP/2+JSON-RPC 2.0 shall be used. HTTP/1.1 bindings shall not be used for agent-to-data or agent-to-tool communication.

5.7 Interface between Network AI Agent and Infrastructure

5.7.1 Interface between Network AI Agent and RAN

5.7.1.1 Requirements

Radio Access Network (RAN) is an important component of mobile network. In 5G systems, the interface between RAN and Core Network (CN) is called N2 [1], which is a point-to-point interface. The N2 interface supports procedures that are related to UE context management, mobility management and is responsible for Non-Access Stratum (NAS) transport.

With the introduction of Network AI Agents into the mobile network architecture, the capabilities of the RAN exposed towards the core network shall be extended in the form of tools for AI agents to discover and invoke. This calls for new procedures for RAN capability discovery, utilization, capability information management (e.g. capability information subscription, un-subscription, and notification), as well as new fields for the interface between RAN and CN.

5.7.1.2 Methods and Data Objects

The following methods shall be supported for the interface between a Network AI Agent and RAN according to the above descriptions.

Table 5.7.1.2-1: Methods for interface exposed by RAN to network AI agent

Method Name	Description	Inputs	Outputs
ran/capability/list	A Network AI Agent requests to query available RAN capabilities of a given type.	params : RanCapabilityListParams[0..1] (see Table 5.7.1.2-2a)	On success: • HTTP Status: 200 OK. • HTTP body contains a "capabilities" list comprised of QueryResult-typed objects as defined in Table 5.7.1.2-2. • An empty list indicates that no capabilities matching the request criteria are currently available.

Method Name	Description	Inputs	Outputs
			On failure: • HTTP body contains a OperationResult-typed object, with "status" field taking the "failed" value, as defined in Table 5.2.2-4.
ran/capability/call	A Network AI Agent requests to invoke a specific RAN capability.	ranCapabilityID : string[1..1], an identifier of RAN capability, as defined in Table 5.7.1.2-2 ranCapabilityParameters: RanCapabilityInfo [1..1] (type-specific parameters per Table 5.7.1.2-2)	On success: • HTTP Status: 200 OK. • HTTP body contains a InvocationResult-typed object with "status" field taking the "success" value, as defined in Table 5.7.1.2-7. On failure: • HTTP body contains a InvocationResult-typed object with "status" field taking the "failed" value.
ran/capability/subscribe	A network AI agent subscribes the real-time changes of interested RAN capabilities.	ranCapabilityID : string[1..1] subscriptionID: string[1..1], a subscription identifier notificationURI: string[1..1], notification address	On success: • HTTP Status: 200 OK. • HTTP body contains a OperationResult-typed object, with "status" field taking the "success" value, as defined in Table 5.2.2-4. On failure: • HTTP body contains a OperationResult-typed object, with "status" field taking the "failed" value, as defined in Table 5.2.2-4.
ran/capability/un-subscribe	A network AI agent un-subscribes the changes of subscribed RAN capabilities.	ranCapabilityID : string[1..1]	On success: • HTTP Status: 200 OK. • HTTP body contains a OperationResult-typed objects with "status" field having the "success" value, as defined in Table 5.2.2-4. On failure: • HTTP body contains a OperationResult-typed object, with "status" field taking the "failed" value, as defined in Table 5.2.2-4.
ran/capability/notify	RAN notifies the updates the capabilities to the subscribed network AI agents.	QueryResult as defined in Table 5.7.1.2-2	NA

RAN is equipped with the capabilities such as sensing, UE paging, UE location, QoS policy enforcement, which can be reported to Network AI Agents upon request. RAN shall indicate the capability identifier, the capability description, capability type and a list of parameters required per capability type. Optionally, it may provide expected output for each capability. The detailed definition for the information fields about RAN capability are shown in the following Table 5.7.1.2-2.

Table 5.7.1.2-2: Fields of the QueryResult data type

Field Name	Type	Presence	Description
ranCapabilityID	string[1..1]	M	This is used to filter the RAN capabilities by the capability type.
ranCapabilityDescription	string[1..1]	M	This describes what the capability does.
ranCapabilityType	enum[1..1]	M	Type discriminator identifying the RAN capability category and thus the applicable parameter schema. Allowed values are "paging", "radio-sensing", "location", "QoS-policy-enforcement". The value of this field determines which fields in ranCapabilityParameters are applicable. (see Table 5.7.1.2-3 through Table 5.7.1.2-6 for per-type definitions).
ranCapabilitySchemaVersion	string[1..1]	M	The schema version for the ranCapabilityParameters structure, expressed as a SemVer 2.0.0 string (major.minor.patch). The value shall not be null or empty.
ranCapabilityParameters	RanCapabilityInfo [1..1]	M	The parameter set for this RAN capability. The applicable fields and their types are determined by the value of ranCapabilityType and capabilitySchemaVersion. See Table 5.7.1.2-3 through Table 5.7.1.2-6 for per-type parameter definitions.
ranCapabilityOutput	string[0..1]	O	Expected output result of this RAN capability.

Table 5.7.1.2-2a: Fields of the RanCapabilityListParams data type

Field Name	Type	Presence	Description
ranCapabilityFilterContent	string[1..1]	M	Value of type string containing RAN capability filter information that may reduce the returned RAN capability information.
ranCapabilityFilterType	enum[1…1]	M	Type of the filtering mechanism to-be-applied. Enumeration: "semantic", "keywords". The value of ranCapabilityFilterType may be used for: • determining the entity that applies the indicated tool filter; • how the value of the ranCapabilityFilterContent field is interpreted. See note.
NOTE: The extension of ranCapabilityFilterType enumeration beyond "semantic" and "keywords" is FFS.			

For each type of capability, the RAN provides the required and optional configurable parameters and their potential values to Network AI Agents. When a Network AI Agent wants to invoke a RAN capability, it shall provide the capability ID and the specific parameter values of this capability.

Table 5.7.1.2-3: Fields of the RanCapabilityInfo when ranCapabilityType is "paging"

Parameter Name	Type	Presence	Value
ueID	string[1..1]	M	UE identifier. Valid values include TMSI or IMSI as defined in ETSI TS 123 003 [9]. Encoding is FFS.
tailist	string[1..1]	M	Representation of the list of tracking areas (PLMN + TAC list) in a format consistent with ETSI TS 123 003 [9]. Encoding is FFS.
drxCycle	enum[0..1]	O	Discontinuous Reception (DRX) cycle. Allowed values are "320ms", "640ms". If absent, the default DRX configuration applies.

Table 5.7.1.2-4: Fields of the RanCapabilityInfo when ranCapabilityType is "radio-sensing"

Parameter Name	Type	Presence	Value
ranSensingSchemesSupp	enum[1..1]	M	Supported sensing schemes. Values include "monostatic", "downlink", "uplink", "multi-static", "passive". At least one value shall be present.
sensingArea	SensingArea[1..1]	M	Geographic sensing area within which this RAN node can perform sensing for this capability, represented as a SensingArea object as defined in Table 5.2.2-11a.
sensingDuration	TimeInterval[0..1]	O	Time interval during which this sensing capability is available or configured, represented as a TimeInterval object as defined in Table 5.2.2-11b. If absent, the capability is considered continuously available subject to operator policy.
sensingAccuracy	number[1..1]	M	Accuracy of the sensing capability, expressed in metres. This value shall be a non-negative real number.

Table 5.7.1.2-5: Fields of the RanCapabilityInfo when ranCapabilityType is "location"

Parameter Name	Type	Presence	Value
ueID	string[1..1]	M	UE identifier. Valid values include SUCI or TMSI as defined in ETSI TS 123 003 [9]. Encoding is FFS.
measurementID	string[1..1]	M	Identifier allocated by the Network AI Agent to distinguish different measurements. This shall not be NULL or empty.
locationSchemes	string[1..*]	M	Supported location schemes. Valid values include "ul-toda", "ul-aoa", "multi-rtt". At least one value shall be present.

Table 5.7.1.2-6: Fields of the RanCapabilityInfo when ranCapabilityType is "QoS-policy-enforcement"

Parameter Name	Type	Presence	Value
QFI	integer[1..1]	M	An integer value
5QI	integer[1..1]	M	An integer value
ARP	integer[1..1]	M	An integer value
GFBR	integer[1..1]	M	An integer value
MFBR	integer[1..1]	M	An integer value
Session-AMBR	integer[1..1]	M	An integer value

After receiving the capability invocation request initiated by a Network AI Agent using ran/capability/call method, the RAN will validate the request and return the status of invocation to indicate whether the capability invocation is successful or not. If it fails, the pre-defined error code and error information should be provided to the Network AI Agent.

Table 5.7.1.2-7: Fields of the InvocationResult

Field Name	Type	Presence	Description
status	enum[1..1]	M	Task execution status. Allowed values are "success" and "failed".
executionResult	Part[0..*]	CM	RAN capability execution result payload, represented as one or more Part objects (see Table 5.6.1.2-4). Present when status is "success".
errorCode	string[0..1]	CM	Machine-readable error code. Present only when status is "failed".
errorMessage	string[0..1]	CM	Human-readable error description. Present only when status is "failed".

To obtain the changes of RAN capability information in real time, the Network AI Agents can subscribe to the RAN capabilities of interest using ran/capability/subscribe method and provide the subscribed ranCapabilityID(s) to the RAN. When the functional description or configurable parameters of the RAN capability changes, the RAN shall notify the subscribed Network AI Agents of the updates using ran/capability/notify method, and providing updated information fields in Table 5.7.1.2-2. If the Network AI Agent is no longer interested in the capabilities of a RAN, it can unsubscribe from the capabilities using ran/capability/un-subscribe method.

5.7.1.3 Protocol

The Next Generation Application Protocol (NGAP), as specified in ETSI TS 138 413 [11], supports the N2 interface between the NG-RAN and the Access and Mobility Management Function (AMF) [1]. Existing NGAP functions include PDU session management, UE context management, UE mobility management, and NAS transport.

In the evolution towards next-generation AI-native mobile communication systems, the NGAP shall be extended to support the interaction between the NG-RAN and Network AI Agents as defined in clause 5.7.1. A new sub-protocol layer shall be introduced, which shall support at minimum the following interactions:

1) RAN Capability Reporting: The RAN shall be able to report its hardware, software, sensing, communication, and computing capabilities to requesting Network AI Agents.

2) Capability Invocation: The Network AI Agent shall be able to trigger or terminate specific RAN-based functions via the capability invocation procedure defined in clause 5.7.1.2.

3) Capability Subscribe/Unsubscribe: The Network AI Agent shall be able to subscribe to event-based or periodic updates regarding RAN-specific capability state changes.

NOTE 1: The detailed NGAP procedures, information elements, and ASN.1 definitions required to support the interactions described in this clause are for further study and are to be specified in the relevant 3GPP specifications. The present document only defines the logical methods and data objects at the Network AI Agent-RAN interface.

NOTE 2: Whether MCP or its derivatives are used as the protocol layer for capability exchange is FFS. The protocol is to be specified in a future version of the present document.

Through these functional enhancements, the network agents can effectively utilize capabilities hosted and exposed by the RAN to meet service requirements in next-generation networks.

5.7.2 Interface between Network AI Agent and Computing Infrastructure

5.7.2.1 Methods and Data Objects

As described in [i.3], Network AI Agents shall provide customized services for UEs and AFs to meet their QoE (Quality of Experience) objectives. Network AI Agents shall deliver customized services through logical networks that are dynamically generated and composed of various network functions, including AI, communication, sensing, and computing functions.

The cloud or computing platform shall provide a communication interface to Network AI Agents to enable agents to discover, invoke, and manage platform capabilities (e.g. deploying, updating, or deleting function instances; configuring parameters and resources; reporting function runtime information). Such an interface may be implemented as a tool-based interface, where the agent discovers and invokes the appropriate tools from the set of available tools exposed by the platform. This interface shall reuse the methods and data objects defined in clause 5.6.2 of the present document.

5.7.2.2 Agent-to-Computing Protocol Requirements

The agent-to-computing-infrastructure interface shall reuse the protocol options defined in clause 5.6.2.3 of the present document.

5.7.2.3 Agent-to-Computing Security Requirements

The agent-to-computing-infrastructure interface shall apply the security requirements defined in clause 5.6.3 of the present document, treating the computing infrastructure platform as the server-side entity and the Network AI Agent as the client.

5.7.3 Security

5.7.3.1 Authentication

Authentication of Network AI Agents accessing the interfaces defined in clause 5.7 of the present document shall comply with clause 5.8.2.4 of the present document. Each Network AI Agent accessing a RAN node or a computing infrastructure platform shall authenticate using mutual TLS (mTLS) with a valid client certificate issued or validated by an operator-trusted certificate authority. A RAN node or computing infrastructure platform shall reject any connection from a Network AI Agent that fails mTLS authentication.

The RAN node or computing infrastructure platform shall present a valid TLS server certificate. The Network AI Agent shall verify this certificate against the operator-trusted certificate authority before transmitting any request.

NOTE: The agent-to-RAN and agent-to-computing-infrastructure interfaces are asymmetric. The Network AI Agent acts as the client and the RAN node or computing infrastructure platform acts as the server.

5.7.3.2 Authorization

5.7.3.2.0 Introduction

Authorization of Network AI Agents shall comply with clause 5.8.3.3 of the present document. A RAN node or computing infrastructure platform shall verify that the requesting agent is authorized to invoke the requested capability prior to processing any request. Authorization shall be based on the authenticated agent identity established during mTLS and the agent's registered profile in the ARF, as specified in clause 5.4.

The following authorization requirements apply per interface:

5.7.3.2.1 Agent-to-RAN Interface

Table 5.7.3.2.1-1 defines the authorization requirements for the five methods of the agent-to-RAN interface.

Table 5.7.3.2.1-1: Authorization requirements for the methods of the agent-to-RAN interface

Method Name	Requirement
ran/capability/list	Requires that the requesting agent holds a valid read authorization for the RAN node. The RAN node shall filter the returned capability list to include only capabilities for which the requesting agent holds at least invocation authorization. A capability discovery response shall not disclose the existence of capabilities to which the requesting agent has no access rights.
ran/capability/call	Requires that the requesting agent holds invocation authorization for the specific capability identified by the ranCapabilityID field. The RAN node shall validate the capability parameters in the request against the parameter schema associated with the capability's ranCapabilityType and ranCapabilitySchemaVersion. It shall reject requests whose parameters do not conform. The RAN node shall additionally support enforcement of operator-configured rate limits on capability invocations per agent.
ran/capability/subscribe	Requires that the requesting agent holds at least read authorization for the RAN node. The RAN node shall validate the notificationURI in the subscription request against the authenticated agent's registered endpoint in the ARF prior to accepting the subscription.
ran/capability/unsubscribe	Requires that the requesting agent is the agent that created the subscription identified by the subscriptionID. The RAN node shall reject cancellation requests from agents that did not originate the subscription.
ran/capability/notify	Shall only be issued by the RAN node. A Network AI Agent receiving a capability notification shall verify that the message originates from the RAN node that accepted the original subscription request, using the server's mTLS identity.

5.7.3.2.2 Agent-to-Computing-Infrastructure Interface

The agent-to-computing-infrastructure interface shall apply the authorization requirements defined in clause 5.6.3.2.2 of the present document, treating the computing infrastructure platform as the tool server and the Network AI Agent as the client. In addition, the computing infrastructure platform shall support enforcement of operator-configured constraints on the categories of platform operations (e.g. NF instantiation, resource scaling, NF deletion) that a given agent is authorized to perform.

5.7.3.3 Data Access Constraints

A Network AI Agent shall not invoke RAN capabilities or computing infrastructure platform operations that involve Class 3 or Class 4 data as defined in clause 5.8.3.5 of the present document, unless explicitly authorized by operator policy. A RAN node or computing infrastructure platform shall enforce these constraints at the capability level, not solely at the interface level. Where a capability invocation result would otherwise contain Class 3 or Class 4 data, the server-side entity shall either redact the data or reject the invocation with a JSONRPCError indicating insufficient authorization.

5.7.3.4 Transport Security

Transport security for the agent-to-RAN and agent-to-computing-infrastructure interfaces shall comply with clause 5.8.4 of the present document. All interactions on these interfaces shall be carried over TLS 1.3 or later. The preferred protocol binding is JSON-RPC 2.0 as specified in clauses 5.7.1.3 and 5.7.2.2. If HTTP/3 is not supported by the deployment environment, HTTP/2+JSON-RPC 2.0 shall be used. HTTP/1.1 bindings shall not be used for agent-to-infrastructure communication.

5.8 Security Requirements

5.8.1 Introduction

The interfaces specified in clause 5 of the present document involve the exchange of sensitive information including user intent, network state data, operator policy data, user subscription data, and infrastructure control commands. The heterogeneous nature of these data classes and the diversity of peer entities (e.g. UEs, third-party Application Functions, other Network AI Agents, data and tool servers, and RAN infrastructure) require differentiated authentication, authorization, and access control mechanisms. This clause specifies the security requirements applicable to all interfaces defined in clause 5. Interface-specific security requirements are specified in clauses 5.2.4, 5.3.4, 5.4.4, 5.5.4, 5.6.3 and 5.7.3 above in the present document.

5.8.2 Authentication

5.8.2.1 General

All communication between a Network AI Agent and a peer entity shall be mutually authenticated prior to the exchange of any application-layer data. Authentication shall be performed using mechanisms appropriate to the peer entity type, as specified in clauses 5.8.2.2 through 5.8.2.5 of the present document.

A Network AI Agent shall not accept a connection from an unauthenticated peer entity. A Network AI Agent shall terminate any connection for which mutual authentication fails without processing any application-layer data.

5.8.2.2 UE Authentication

Authentication of UEs shall be performed using the security mechanisms defined in 3GPP TS 33.501 [5] and subsequent standards. NAS-level message integrity protection and ciphering, as specified in 3GPP TS 24.501 [2], shall be applied to all NAS messages carrying intent-based service requests as defined in clause 5.2.3.

NOTE: The authentication of an UE AI Agent mediating UE-side interactions (see clause 5.2.1) is treated as equivalent to UE authentication for the purposes of the present clause. The distinction between UE platform and UE AI Agent authentication is internal to the UE implementation and outside the scope of the present document.

5.8.2.3 Third-Party Application Function (AF) Authentication

Authentication of third-party Application Functions shall use mutual Transport Layer Security (mTLS) with certificates issued or validated by an operator-trusted certificate authority. Where the AF interacts with the Network AI Agent via the Network Exposure Function (NEF), the authentication and credential validation procedures apply [5].

An AF shall present a valid client certificate during TLS handshake. A Network AI Agent shall reject connections from AFs presenting certificates that are expired, revoked, or not issued by a trusted certificate authority.

Where both direct AF-to-agent and NEF-mediated interactions are supported, the same trust anchors and certificate validation policies shall apply.

5.8.2.4 Network AI Agent Authentication

Authentication between Network AI Agents shall use mutual TLS (mTLS) with certificates issued by the operator or by an operator-trusted certificate authority. Authentication may also be performed based on the authentication method included in the Decentralized Identifier (DID) document if a World Wide Web consortium (W3C®) DID scheme is used for agent identity. A Network AI Agent shall verify that the peer agent's certificate corresponds to a currently registered entry in the ARF before accepting any task request. An agent whose ARF registration has expired or been deregistered shall not be accepted as a peer.

5.8.2.5 Data and Tool Server Authentication

Authentication of data resource servers and tool servers (as defined in clauses 5.6.1 and 5.6.2) shall use mTLS. Tool and data servers shall present certificates issued by a certificate authority trusted by the operator. A Network AI Agent shall verify server certificates before submitting any data query or tool invocation request.

5.8.3 Authorization

5.8.3.1 General

Authorization shall be enforced independently of authentication. Successful authentication is a necessary but not sufficient condition for permitting access to a specific interface method or data resource.

Authorization decisions shall be performed on a per-request basis based on the identity of the peer entity, the requested method, the data sensitivity class of the resource being accessed, and the operator's policy and local regulations. The data sensitivity classes are defined in clause 5.8.3.5.

5.8.3.2 AF Authorization

Authorization of third-party Application Functions shall be performed using the OAuth 2.0 authorization framework, with the operator's NEF acting as the authorization server, consistent with the 3GPP service-based interface exposure framework defined in 3GPP TS 23.501 [1]. An AF shall obtain an access token scoped to the specific Network AI Agent service it is authorized to access. A Network AI Agent shall validate the AF's access token before processing any afService/create, afService/update, afService/get, or afService/delete request.

The scope of the access token is defined as follows:

- it shall identify the authorized AF (by AF ID);
- it shall identify the authorized service type (by method name); and
- it shall identify the maximum data sensitivity class accessible to the AF (see clause 5.8.3.5).

An AF shall not be granted access to operator policy data, user subscription data, or network knowledge bases via the interface defined in clause 5.3.

5.8.3.3 Network AI Agent Authorization

Inter-agent authorization shall be enforced by validating that the requesting agent holds a valid ARF registration and that the requested task type is within the scope of the requesting agent's registered capabilities. A Network AI Agent shall reject task requests from agents whose registered capabilities do not include the requested task type.

5.8.3.4 UE Authorization

A Network AI Agent shall authorize UE-originated service requests by verifying the UE's identity (e.g. SUPI, W3 DID) against the operator's subscription data. Capability configuration requests (UECapabilityconfig, as defined in clause 5.2.2) shall only be processed following explicit user consent and preferences validated during service initiation, and recorded in the operator's subscription data, as specified in clause 5.2.1.

5.8.3.5 Data Sensitivity Classes

Data resources accessible to Network AI Agents are classified into the following sensitivity classes for the purposes of access control:

1) Class 1 - Public network information: network topology, publicly advertised service descriptions, and non-subscriber-specific performance metrics.

2) Class 2 - Operational data: network state information, resource usage metrics, aggregate analytics, and log data with personal information removed.

3) Class 3 - Subscriber data: user subscription information, UE location data, SUPI-linked records, and per-user QoS policies.

4) Class 4 - Operator policy data: network operator configuration data, security policies, and slice management parameters.

Access to Class 3 and Class 4 data by a Network AI Agent shall require user consent, user preference, and explicit operator authorization, except where otherwise mandated by applicable law or regulation. Third-party AFs shall not be granted access to Class 3 or Class 4 data via the interfaces defined in the present document.

5.8.4 Transport Security

All application-layer interfaces specified in clause 5, with the exception of the NAS-based UE interface specified in clause 5.2.3, shall be carried over TLS 1.3 or later. TLS 1.2 may be used only where TLS 1.3 is not supported by the peer entity's deployment environment; in such cases, the cipher suites permitted shall be restricted to those recommended in NIST SP 800-52 [i.19].

NOTE: A cipher suite specifies a collection of algorithms for key exchange and for providing confidentiality and integrity services to application data. The cipher suite negotiation occurs during the TLS handshake protocol [i.19].

The NAS-based UE interface specified in clause 5.2.3 shall use the NAS security mechanisms defined in 3GPP TS 33.501 [5] and 3GPP TS 24.501 [2].

5.8.5 Access Control for the Agent Repository Function

The ARF is the trust anchor for agent identity in the interfaces defined in the present document. The following access control requirements apply to the ARF interface specified in clause 5.4:

1) Agent registration (agent/register) shall be authenticated using mTLS. Only operator-authorized entities shall be permitted to register a new Network AI Agent.

2) Agent update (agent/update) shall only be performed by the registered agent itself or by an operator-authorized management entity.

3) Agent deregistration (agent/deregister) shall only be performed by the registered agent itself, or by an operator-authorized management entity, or automatically upon expiry of the AgentProfile Expiration Time field.

4) Agent discovery (agent/discover) shall be accessible to any authenticated and authorized Network AI Agent, UE AI Agent, and other entity. Discovery responses shall not include Class 3 or Class 4 data in agent profile fields.

5.8.6 Auditability and Traceability

Consistent with the Traceability design principle stated in clause 4.3, all interface interactions defined in clause 5 shall be logged by the Network AI Agent. Log records shall include, at a minimum:

1) the identity of the peer entity (UE ID, AF ID, Agent ID, or resource URI as applicable);

2) the method invoked;

3) a timestamp;

4) the outcome (success or failure, with error code if applicable); and

5) for data access methods, the data sensitivity class of the accessed resource.

To enhance traceability, log records may also include a cryptographic signature generated by Network AI Agent. To enhance security, log records may be cryptographically protected. In addition, operations on log records shall be restricted by access control policies. Only authenticated and authorized entity shall be able to Create, Read, Update, or Delete (CRUD) log records.

Log records shall be retained in accordance with applicable regulatory requirements and operator policy. Log data shall be classified as Class 4 data for access control purposes.

5.8.7 Relationship to Referenced Security Frameworks

The security requirements in the present clause are consistent with the following referenced frameworks:

1) 3GPP TS 33.501 [5]: security architecture and procedures for the 5G system, covering NAS security, SBI security, and network exposure security.

2) NIST SP 800-53 [i.17]: security and privacy controls for information systems, applicable to the access control, audit, and identification and authentication control families.

3) NIST SP 800-207 [i.18]: Zero Trust Architecture, applicable to the per-request authorization model used for inter-agent and AF-to-agent interactions.

6 Communication Examples

6.1 Agent Interface Communication Examples

The following clauses present illustrative examples of message interactions for the interfaces defined in clause 5 of the present document. Each example depicts a specific sequence of method invocations between a Network AI Agent and a peer entity, using the methods and data objects defined in clauses 5.5, 5.6 and 5.7. The examples are all informative and do not constitute normative requirements.

6.2 Agent-to-Tool Interaction Example

Figure 6.2-1 illustrates an example interaction between a Network AI Agent, operating as a tool client, and an external tool server located in a mobile core network. This uses the agent-to-tool interface specified in clause 5.6.2. The example illustrates a tool discovery followed by a tool invocation. The Network AI Agent sends a tool discovery request indicating the "tools/list" method. This message does not contain any input parameters, thus the "params" field is omitted in the request. The example assumes JSON-RPC 2.0 serialization over HTTP/2 or HTTP/3 as described in clause 5.6.2.3.

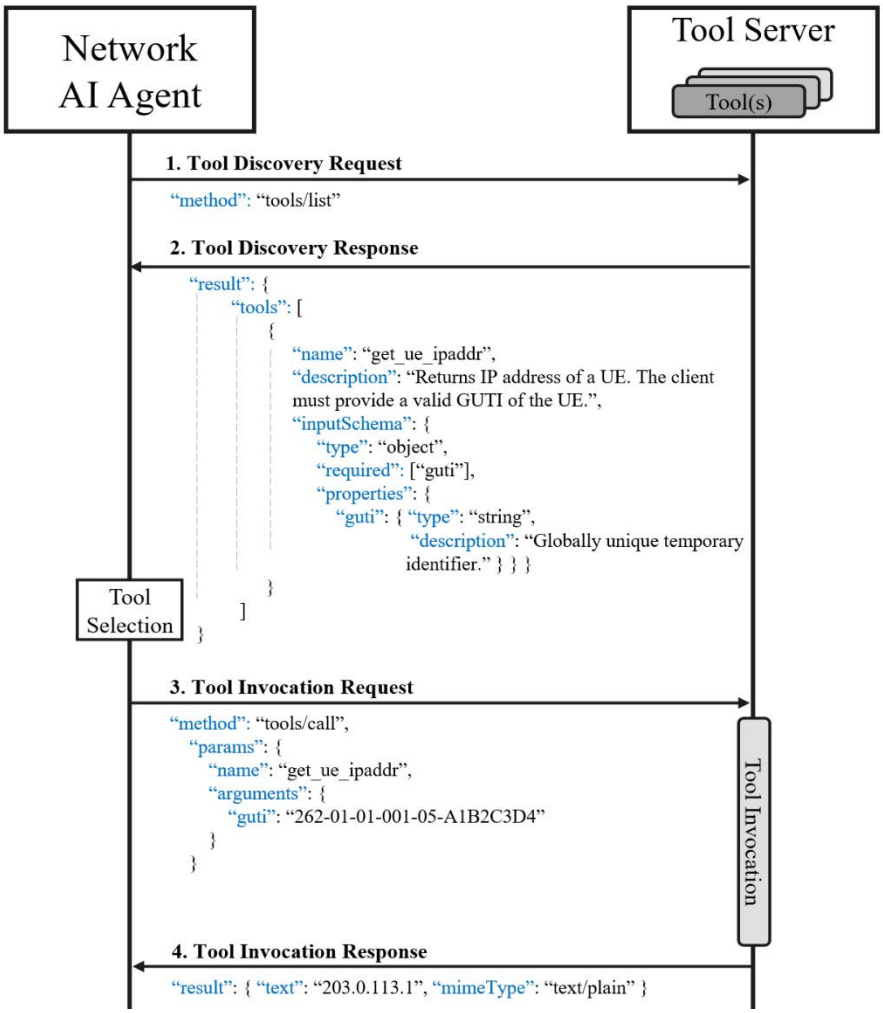


Figure 6.2-1: Example agent-to-tool interaction including discovery and invocation

In step 1, the Network AI Agent discovers the tools exposed by the tool server by invoking the tool/list method, as defined in Table 5.6.2.2-1. The JSON-RPC request object sets the method member to "tool/list" and omits params, indicating an unfiltered discovery request.

In step 2, the tool server responds with a list of ToolInfo objects as defined in Table 5.6.2.2-2. Each ToolInfo object includes the following data as defined in Table 5.6.2.2-2:

- name, a unique identifier for the tool;
- description, a human-readable description of the tool's purpose;
- inputSchema, a SchemaObject, including the properties and required fields describing the tool's input arguments, as defined in Table 5.6.2.2-7;
- optionally outputSchema, a SchemaObject describing the structure of the tool's output, as defined in Table 5.6.2.2-7.

The example in Figure 6.2-1 shows a "get_ue_ipaddr tool". Its inputSchema is of type SchemaObject with:

- type = "object";
- properties containing a single SchemaPropertyObject for the "guti" argument (see Table 5.6.2.2-8);
- required containing the value "guti", indicating that this argument is mandatory.

In step 3, the Network AI Agent invokes the tool by sending a tool/call request. The request sets the method member to "tools/call" and the params member to a ToolCallParams object as defined in Table 5.6.2.2-3, including:

- name = "get_ue_ipaddr";
- arguments, a JSON object whose structure conforms to the inputSchema returned in step 2 (e.g. {"guti": "<GUTI value>"}).

In step 4, the tool server returns a response as defined in Table 5.6.2.2-1. In the example, the result contains a Part object as defined in Table 5.6.1.2-4, with either the text field set to the string representation of the UE IP address and mimeType = "text/plain", or the raw field set to the encoded bytes and an appropriate MIME type.

The figure omits JSON-RPC envelope fields such as jsonrpc and id, as well as HTTP header fields, for brevity. Implementations shall follow the complete JSON-RPC 2.0 specification [3] and the HTTP binding requirements specified in clause 5.6.2.3.

6.3 Agent-to-Data Interaction Example

The message exchange depicted in Figure 6.3-1 showcases a Network AI Agent interacting with a data server that resides in a mobile network using the agent-to-data interface defined in clause 5.6.1. The messaging sequence consists of two main parts:

1) discovery phase;
2) data retrieval phase.

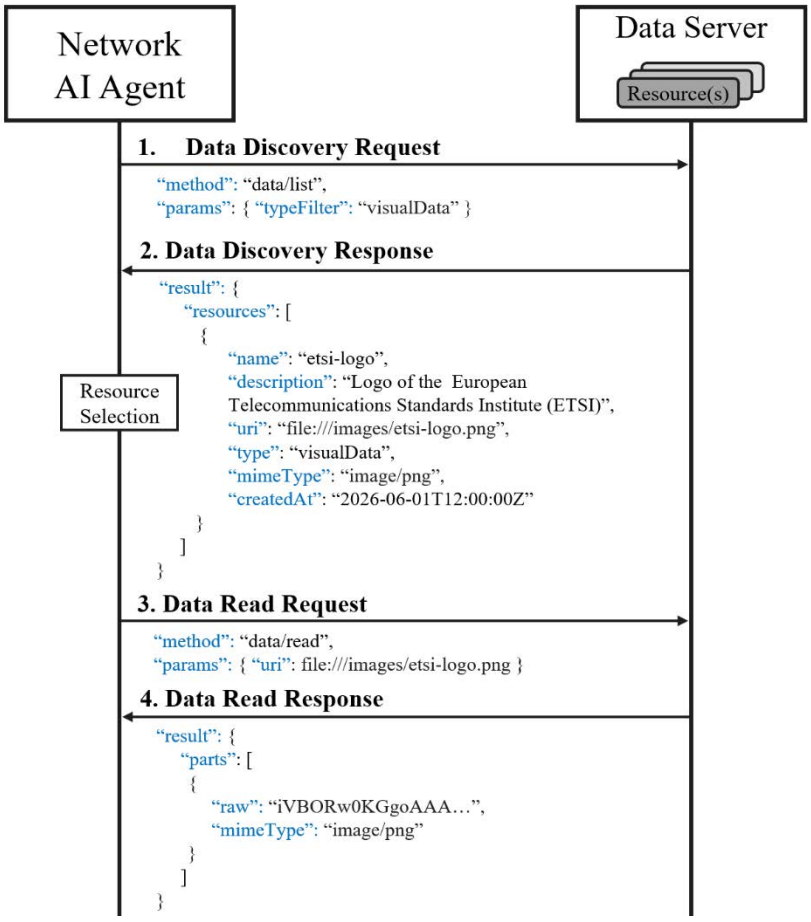


Figure 6.3-1: Example agent-to-data interaction including resource discovery and resource retrieval

In step 1 of the discovery phase, the Network AI Agent issues a data/list request as defined in Table 5.6.1.2-1. The JSON-RPC request object sets method to "data/list" and includes a ResourceReqParams object in the params member as defined in Table 5.6.1.2-2. In the example, ResourceReqParams contains:

- typeFilter with a single value "visualData", indicating that only visual data resources (e.g. images, videos) shall be returned;
- optionally semanticFilter and metadata, which are omitted in the example.

In step 2, the data server responds with HTTP status 200 OK and a JSON-RPC result containing a list of DataInfo objects as defined in Table 5.6.1.2-3. Each DataInfo object includes:

- uri, a unique resource identifier formatted as a URI per IETF RFC 3986 [7];
- type = "visualData", consistent with the typeFilter in the request;
- name, an optional human-readable name such as "etsi-logo";
- description, a mandatory human-readable description;
- createdAt, the creation time expressed as an ISO 8601 [14] UTC datetime string;
- optionally modifiedAt, mimeType, and metadata.

For presentation purposes, the example shows a single DataInfo entry representing a PNG image resource named "etsi-logo" with mimeType = "image/png".

In step 3, the Network AI Agent selects the desired resource and sends a data/read request as defined in Table 5.6.1.2-1. The JSON-RPC request sets method to "data/read" and passes the uri obtained from the discovery response as a string parameter. The uri shall not be NULL or empty and shall match one of the URIs returned by data/list.

In step 4, the data server responds with HTTP status 200 OK and a JSON-RPC result containing one or more Part objects as defined in Table 5.6.1.2-4. In the example, the response body includes a single Part object with:

- raw containing a Base64-encoded representation of the PNG image bytes;
- mimeType = "image/png".

The consumer can reconstruct the binary content by decoding the Base64-encoded raw field. Where the data resource is of type textData, the server would instead populate the text field of Part and set mimeType = "text/plain" or another appropriate textual MIME type.

Figure 6.3-1 illustrates only the relevant JSON-RPC payload elements and omits HTTP header fields and JSON-RPC envelope members such as jsonrpc and id. Implementations shall follow the JSON-RPC 2.0 specification [3] and the transport security requirements specified in clause 5.6.3.4.

6.4 Agent-to-Agent Interaction Example

Figure 6.4-1 shows an example agent-to-agent interaction including task initiation and task status query, using the agent-to-agent interface specified in clause 5.5. In this example, Network AI Agent 1 acts as the client agent and Network AI Agent 2 acts as the server agent. The example assumes that JSON-RPC 2.0 is used for data serialization in message payload.

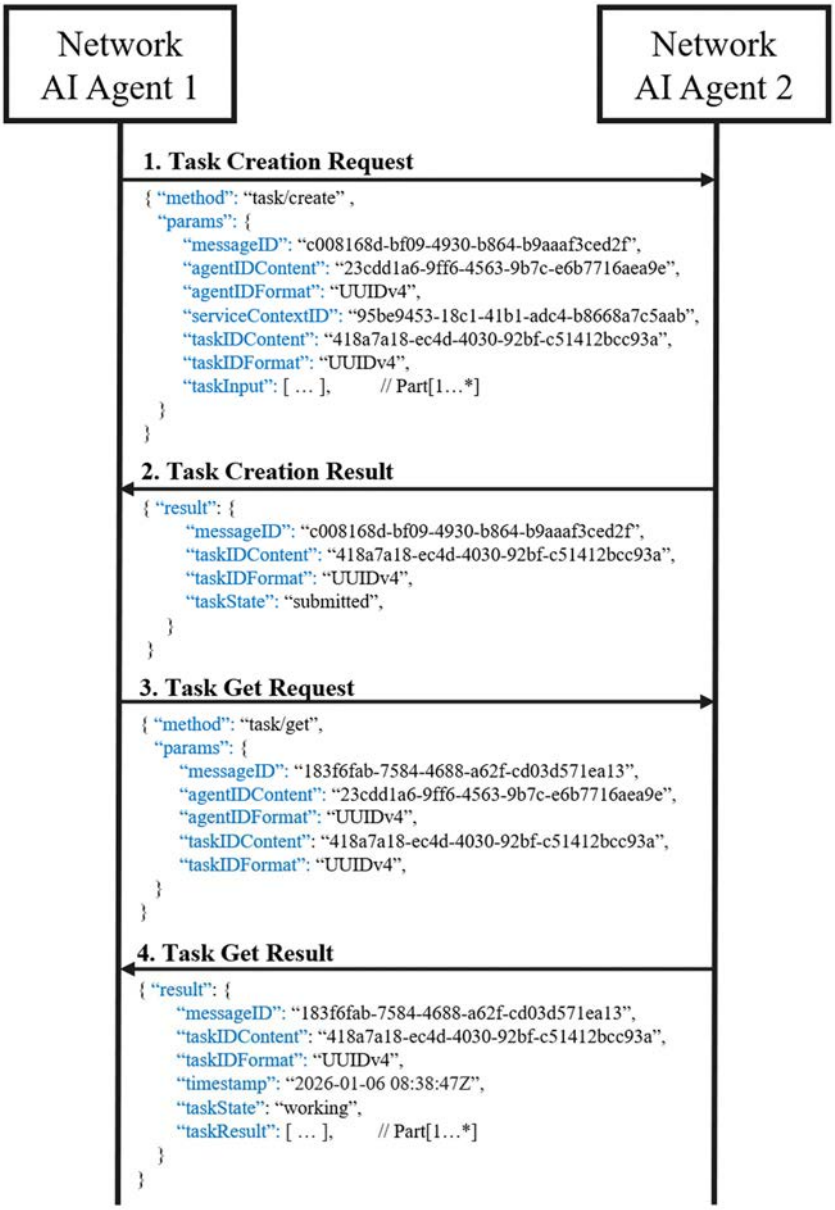


Figure 6.4-1: Example agent-to-agent interaction including task initiation and task status query

In step 1, Network AI Agent 1 initiates a new task by sending a task/create request, as defined in Table 5.5.2-1. The request payload includes at least:

- messageID, a per-request correlation identifier;
- agentID, the composite identifier of Network AI Agent 1, expressed in UUIDv4 format as specified in clause 5.5.2;
- taskID, a composite client-generated task identifier that is unique within the scope of the agent;
- serviceContextID, binding the task to the originating service request as defined in Table 5.5.2-2;
- taskInput, a list of one or more Part objects as defined in Table 5.6.1.2-4, encoding task description and any additional input data required to complete the task.

In step 2, Network AI Agent 2 responds with a JSON message containing:

- messageID, echoing the value from the corresponding task/create request;
- taskID, echoing the created composite task identifier;

- timestamp, an ISO 8601 [14] UTC datetime string representing the time at which the task was accepted;
- taskStatus, an instance of the TaskStatus enumeration defined in Table 5.5.2-7. In the example, the immediate task state is "submitted", indicating that the task has been accepted but processing has not yet started.

In step 3, Network AI Agent 1 queries the state of the previously created task by sending a task/get request as defined in Table 5.5.2-1. The request payload includes:

- messageID, a new correlation identifier for the query;
- agentID, identifying the querying agent;
- taskID, identifying the task whose status is requested.

In step 4, Network AI Agent 2 responds with a JSON message containing:

- messageID, echoing the messageID from the task/get request;
- taskID, echoing the queried task identifier;
- timestamp, indicating the time of the status evaluation in ISO 8601 [14] UTC format;
- taskStatus, an instance of the TaskStatus enumeration defined in Table 5.5.2-7;
- optionally taskResult, a list of Part objects containing partial or final results generated for the task, depending on the current state.

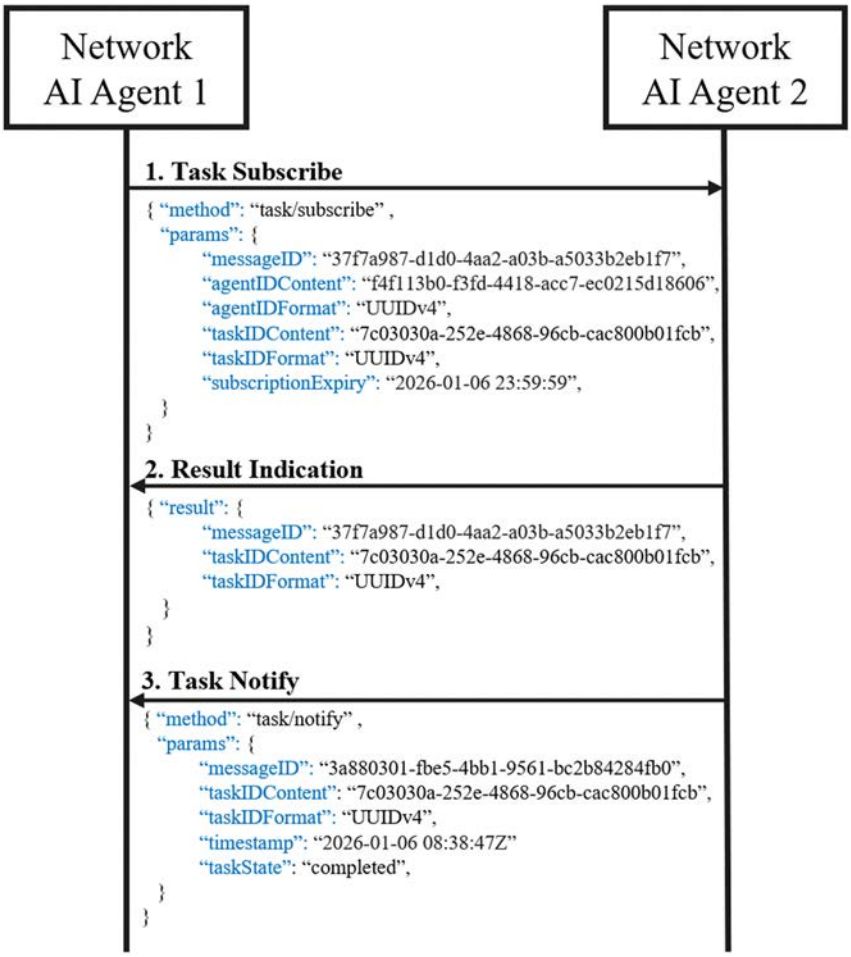


Figure 6.4-2: Example agent-to-agent interaction including subscription to receiving task notifications

Figure 6.4-2 shows an example interaction in which Network AI Agent 1 subscribes to task notifications from Network AI Agent 2 using the task/subscribe and task/notify methods defined in Table 5.5.2-1. The example assumes that JSON-RPC 2.0 is used for data serialization.

In step 1, Network AI Agent 1 invokes task/subscribe with a payload containing:

- messageID, a correlation identifier;
- agentID, identifying the subscribing agent;
- taskID, identifying the task to be monitored;
- subscriptionExpiry, expressed as an ISO 8601 [14] UTC datetime string indicating the subscription lifetime;
- optionally notifyOnState, which is omitted in the example. When notifyOnState is omitted, notifications occur at each state transition as defined in Table 5.5.2-5a.

In step 2, Network AI Agent 2 responds with a direct result indicating acceptance or rejection of the subscription request, including status and, on success, a subscription identifier if defined in clause 5.5.2.

In step 3, after observing a status change in the subscribed task, Network AI Agent 2 sends a task/notify request to Network AI Agent 1. The notification message includes:

- messageID, generated by Network AI Agent 2;
- taskID, identifying the task whose state has changed;
- timestamp, expressing the time of the state transition as an ISO 8601 [14] UTC datetime string;
- taskStatus, indicating the new task state (e.g. "completed").

This example illustrates the use of task/subscribe and task/notify for event-driven monitoring of task progress between Network AI Agents. Implementations shall ensure that notifications are subject to appropriate authentication and authorization controls as specified in clause 5.5.4.

7 Areas for Future Study

7.1 Open issues for the present document

Void.

NOTE: All issues identified as For Further Study (FFS) in the body of the present document are collected in clause 7.2.

7.2 Issues for Future Study

The following are issues for further study in a future release of the present document.

From clause 4.4:

1) The present document specifies only the internal interfaces of the agent-based Assisted System. The specification of ENI External Reference Points (ERPs) and the ENI API Broker for this Assisted System is deferred to a future work item.

2) The detailed definition of how an ENI System observes, manages, and adapts the agent-based Assisted System via ERPs is for further study.

From clause 5.2.1, note 2:

1) ETSI GS ENI 019 [8] defines the ENI Extended Core information model. Extensions to ETSI GS ENI 019 [8] to better support interfaces and context representations defined in the present document are for further study in Release 4 of ETSI GS ENI 019 [8].

From the note below Table 5.2.2-11:

1) The interval [startTime, endTime] is inclusive at both endpoints. Representation of recurring or non-contiguous time intervals is for further study.

From clause 5.3.3:

1) The present document considers HTTP/1.1+JSON-REST, HTTP/1.1+JSON-RPC 2.0, HTTP/2+gRPC, and HTTP/3+gRPC as candidate transport bindings. A detailed analysis of alternative transport protocols, including performance, header compression, and deployment constraints in mobile networks, is for further future study in a future release of the present document.

From clause 5.4.2:

1) Handling of abandoned registrations (i.e. expired registrations that are neither refreshed nor explicitly deregistered) is for further study.

From clause 5.6.1.2:

1) The agent-to-data interface supports attribute- and type-based discovery of data resources. A detailed analysis of existing semantic matching algorithms for data resource discovery, including their advantages, limitations, and implementation challenges in mobile networks, is for further study in a future release of the present document.

From clause 5.6.2.2:

1) The agent-to-tool interface supports discovery of tools based on capabilities and schemas. A detailed analysis of existing semantic matching algorithms for tool discovery, including their advantages, limitations, and implementation challenges in mobile networks, is for further study in a future release of the present document.

From clause 5.7.1.3

1) Two alternative approaches for the agent-to-RAN interface are considered:

   a) extending NGAP with an additional functional sub-layer and associated ASN.1 definitions to support the RAN capability interactions defined in clause 5.7.1; and

   b) using the protocol options listed in clause 5.3.3 (e.g. JSON-RPC or gRPC over HTTP) in conjunction with an ENI-based access path, without NGAP extensions.

   The detailed definition, evaluation, and standardization of the selected protocol binding(s), including message types and information elements, are for further study in cooperation with the relevant 3GPP groups.

From clause 5.7.1.3, note 2:

1) Whether the Model Context Protocol (MCP) or its derivatives are used as the protocol layer for capability exchange between Network AI Agents and the RAN is for further study. This includes investigation of MCP's suitability for NG-RAN environments, security implications, and alignment with existing 3GPP procedures.

History

Version	Date	Status
V4.1.1	September 2026	Publication